Introduction to Deep Neural Networks

Jefersson A. dos Santos (j.santos@sheffield.ac.uk)

Department of Computer Science
The University of Sheffield

Outline

1. Introduction
   - Deep Learning, Artificial Intelligence, Machine Learning, etc
2. Basic Concepts
   - Perceptron
   - Loss function
   - Activation functions
   - Training strategies
3. Deep Neural Networks
   - Multi-Layer Perceptrons (MLPs)
   - Backpropagation
4. Conclusions
5. PyTorch

Learning Outcomes

By the end of this lecture, you should be able to:

- Discuss about differences between deep and other ML-based methods
- Recognize basic concepts related to neural networks
- Recognize structures of an artificial neural network
- Understand the training process of a multilayer neural network

What is Deep Learning?

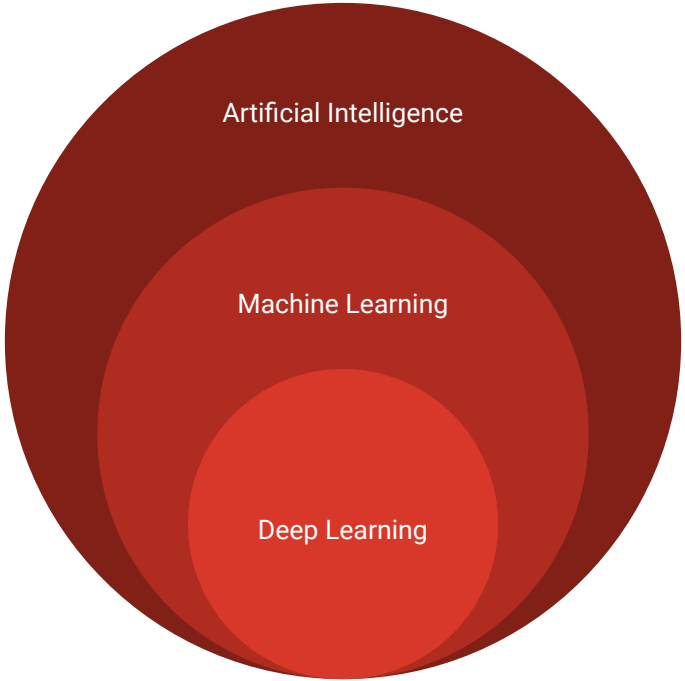
Introduction

What is Deep Learning?

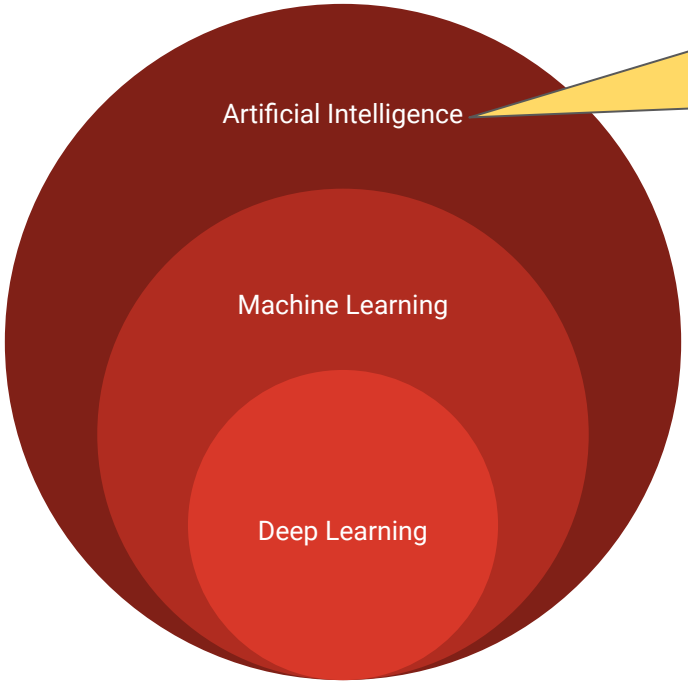
https://www.menti.com/alvu4w5giar6

Results: https://www.mentimeter.com/app/presentation/alcu9edxz63k2tj2c7f439wxy3vipqtg

Introduction



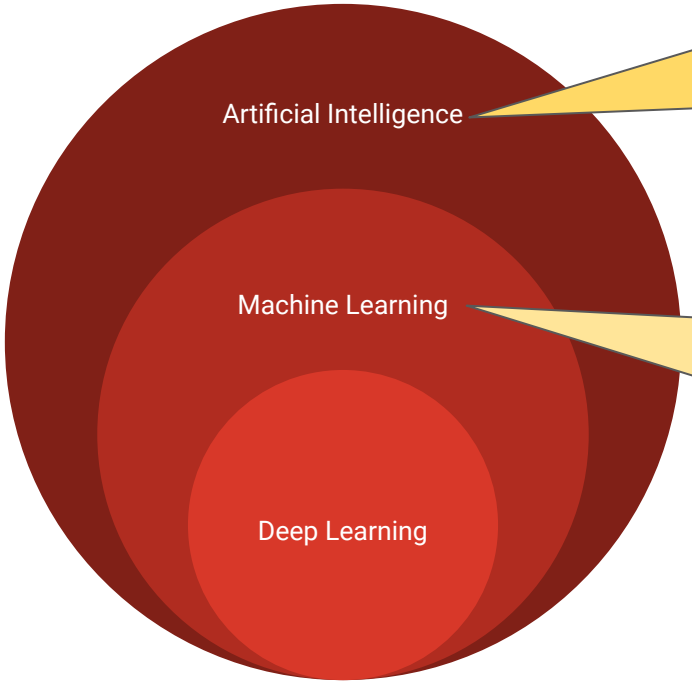
Introduction



Techniques that enables computers to mimic human intelligence

AI encompasses a broad range of techniques and approaches aimed at enabling machines to perform tasks that typically require human intelligence, such as reasoning, problem-solving, learning, perception, and natural language understanding.

Introduction



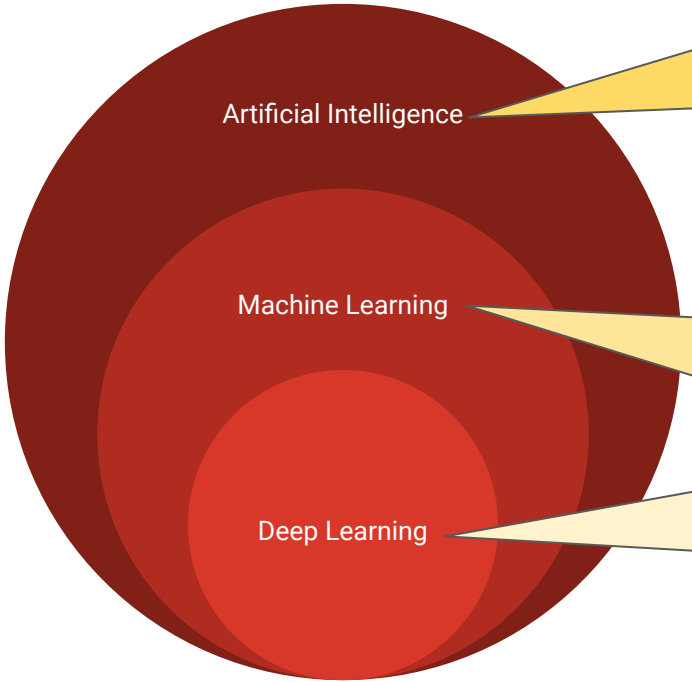
Techniques that enables computers to mimic human intelligence

AI encompasses a broad range of techniques and approaches aimed at enabling machines to perform tasks that typically require human intelligence, such as reasoning, problem-solving, learning, perception, and natural language understanding.

Learning from Data

ML is a subset of AI that focuses on developing algorithms and statistical models that allow computers to learn and improve from experience without being explicitly programmed. ML algorithms can analyze data, recognize patterns, and make predictions or decisions based on that data.

Introduction



Techniques that enables computers to mimic human intelligence

AI encompasses a broad range of techniques and approaches aimed at enabling machines to perform tasks that typically require human intelligence, such as reasoning, problem-solving, learning, perception, and natural language understanding.

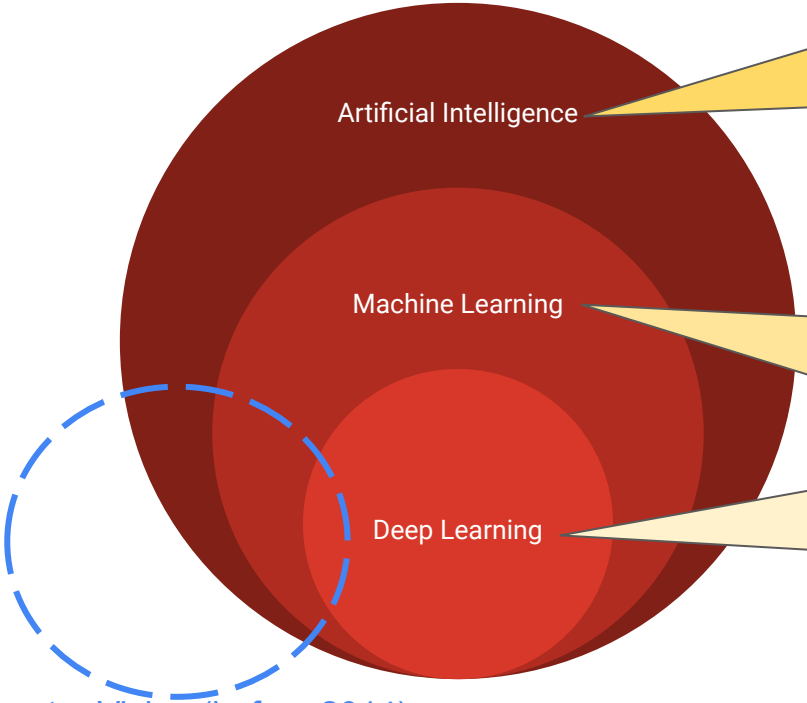
Learning from Data

ML is a subset of AI that focuses on developing algorithms and statistical models that allow computers to learn and improve from experience without being explicitly programmed. ML algorithms can analyze data, recognize patterns, and make predictions or decisions based on that data.

Neural networks with multiple hidden layers

DL is a subfield of machine learning that uses neural networks with multiple layers (deep architectures) to extract high-level features from raw data. Deep learning algorithms are particularly effective for tasks involving large amounts of unstructured data, such as images, text, and audio.

Introduction



Techniques that enables computers to mimic human intelligence

AI encompasses a broad range of techniques and approaches aimed at enabling machines to perform tasks that typically require human intelligence, such as reasoning, problem-solving, learning, perception, and natural language understanding.

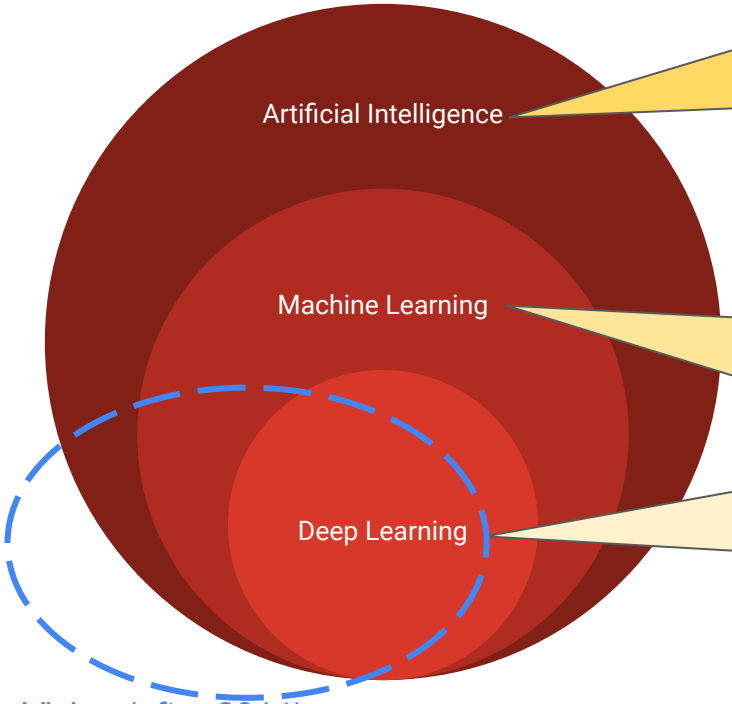
Learning from Data

ML is a subset of AI that focuses on developing algorithms and statistical models that allow computers to learn and improve from experience without being explicitly programmed. ML algorithms can analyze data, recognize patterns, and make predictions or decisions based on that data.

Neural networks with multiple hidden layers

DL is a subfield of machine learning that uses neural networks with multiple layers (deep architectures) to extract high-level features from raw data. Deep learning algorithms are particularly effective for tasks involving large amounts of unstructured data, such as images, text, and audio.

Introduction



Techniques that enables computers to mimic human intelligence

AI encompasses a broad range of techniques and approaches aimed at enabling machines to perform tasks that typically require human intelligence, such as reasoning, problem-solving, learning, perception, and natural language understanding.

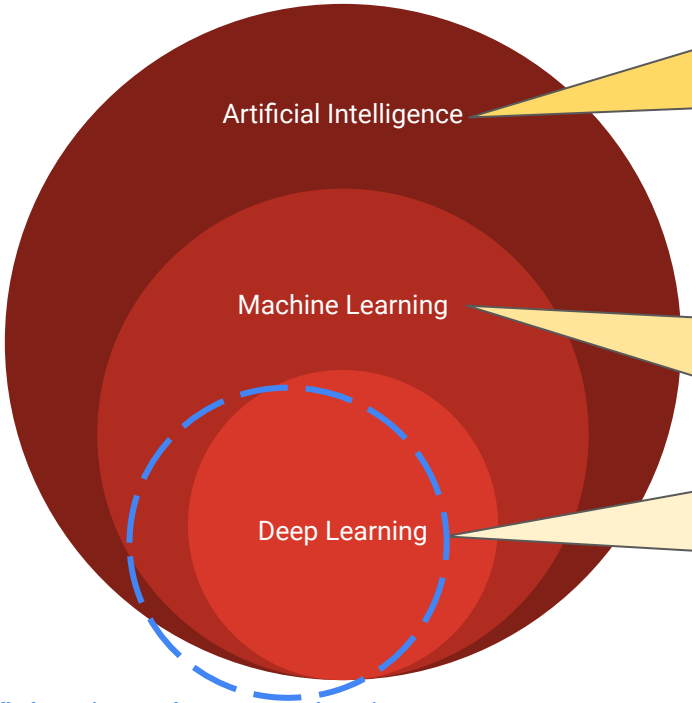
Learning from Data

ML is a subset of AI that focuses on developing algorithms and statistical models that allow computers to learn and improve from experience without being explicitly programmed. ML algorithms can analyze data, recognize patterns, and make predictions or decisions based on that data.

Neural networks with multiple hidden layers

DL is a subfield of machine learning that uses neural networks with multiple layers (deep architectures) to extract high-level features from raw data. Deep learning algorithms are particularly effective for tasks involving large amounts of unstructured data, such as images, text, and audio.

Introduction



Computer Vision (novel approaches)

Techniques that enables computers to mimic human intelligence

AI encompasses a broad range of techniques and approaches aimed at enabling machines to perform tasks that typically require human intelligence, such as reasoning, problem-solving, learning, perception, and natural language understanding.

Learning from Data

ML is a subset of AI that focuses on developing algorithms and statistical models that allow computers to learn and improve from experience without being explicitly programmed. ML algorithms can analyze data, recognize patterns, and make predictions or decisions based on that data.

Neural networks with multiple hidden layers

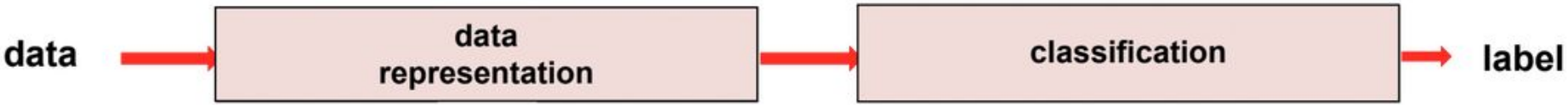
DL is a subfield of machine learning that uses neural networks with multiple layers (deep architectures) to extract high-level features from raw data. Deep learning algorithms are particularly effective for tasks involving large amounts of unstructured data, such as images, text, and audio.

What makes deep learning special?

Classical Image Analysis

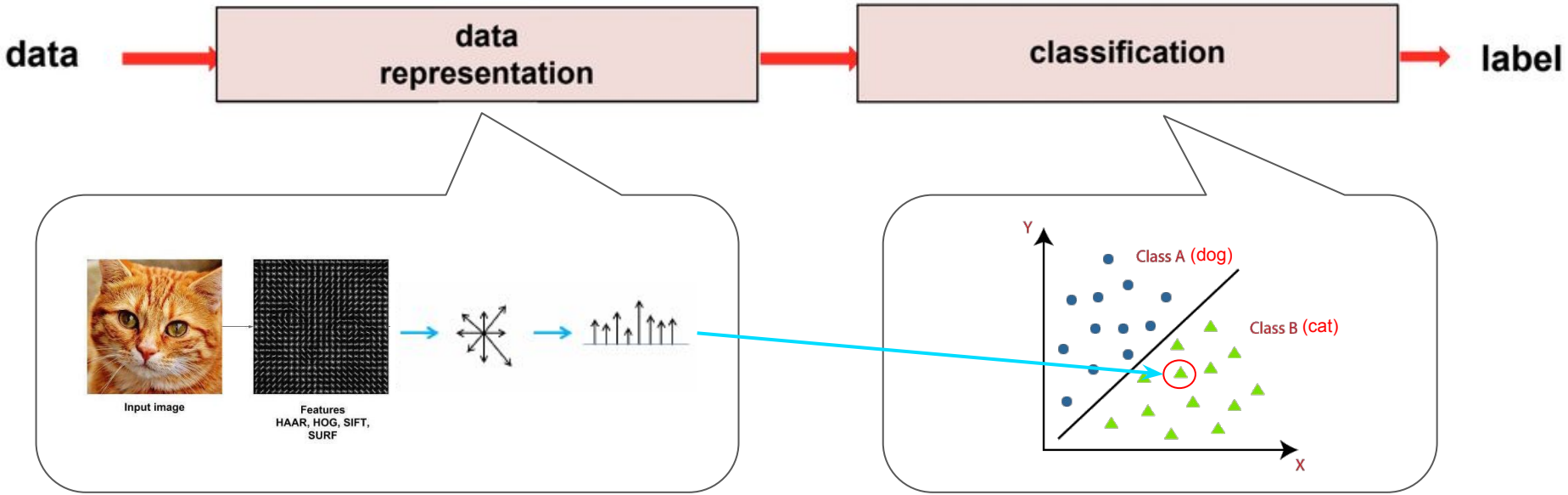
What makes deep learning special?

Classical Image Analysis + ML



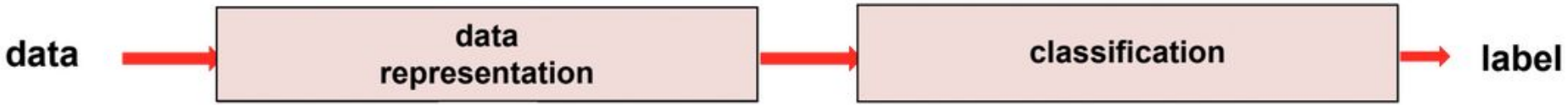
What makes deep learning special?

Classical Image Analysis + ML



What makes deep learning special?

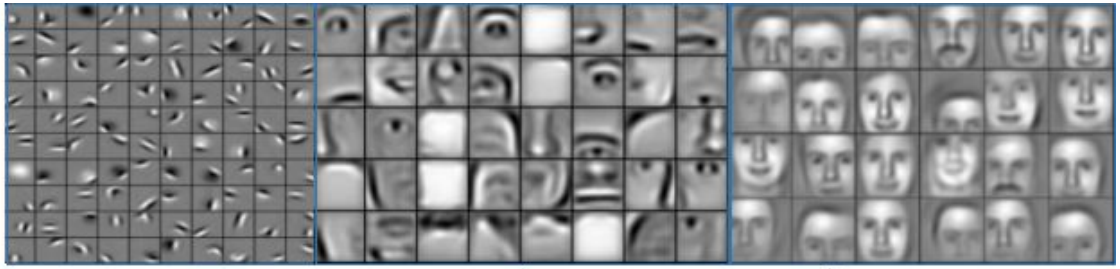
Classical Image Analysis

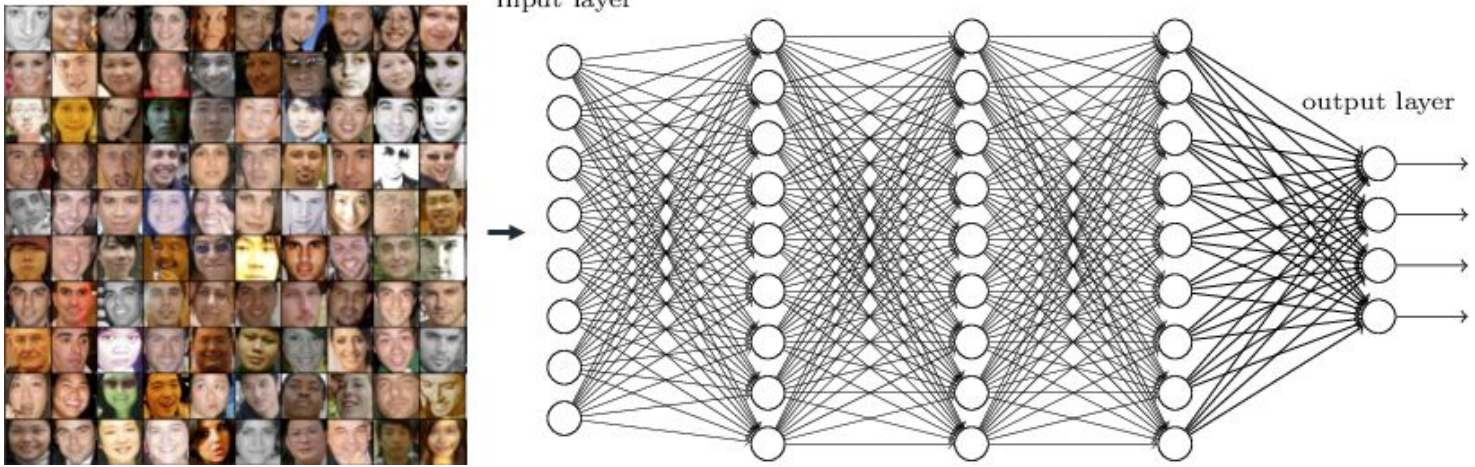


Deep Learning

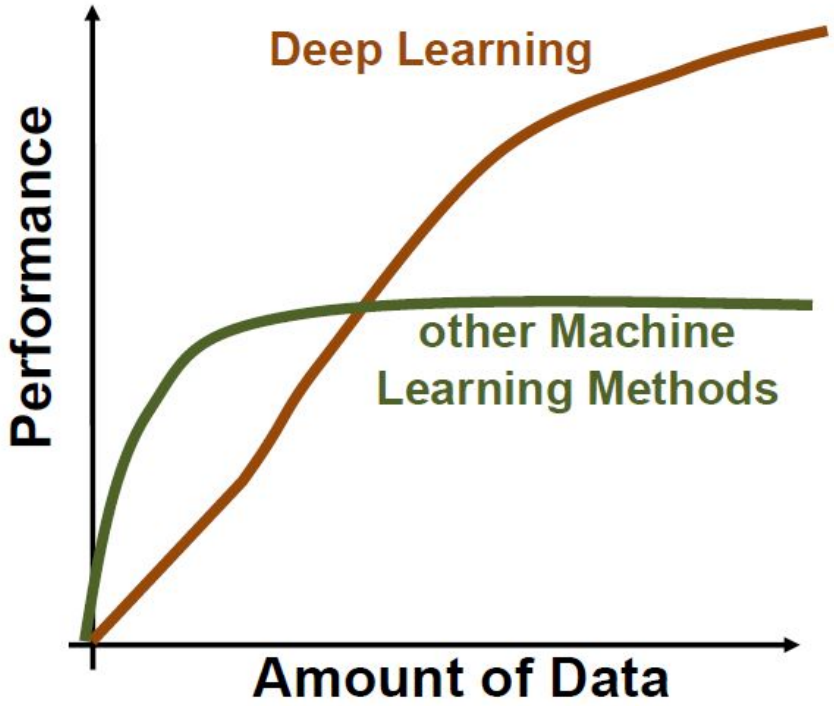
What makes deep learning special?

Deep neural networks learn hierarchical feature representations



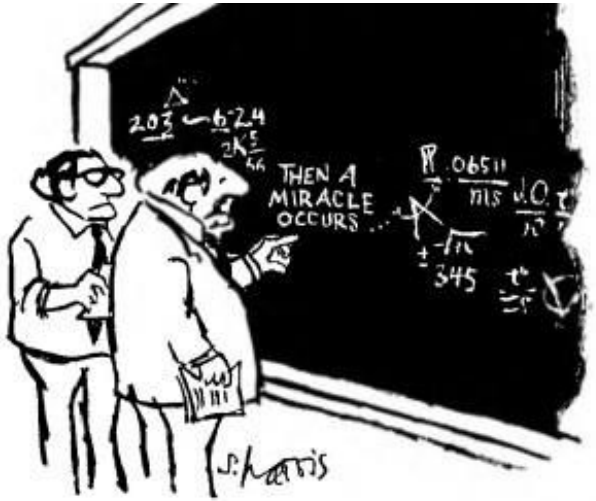


What makes deep learning special?



But it is still have some limitations…

Interpretability



"I THINK YOU SHOULD BE MORE EXPLICIT HERE IN STEP TWO."

"Neural networks are able to learn, but they do not tell anyone what they have learned".

But it is still have some limitations…

Hardware Dependency

Artificial Neural Networks

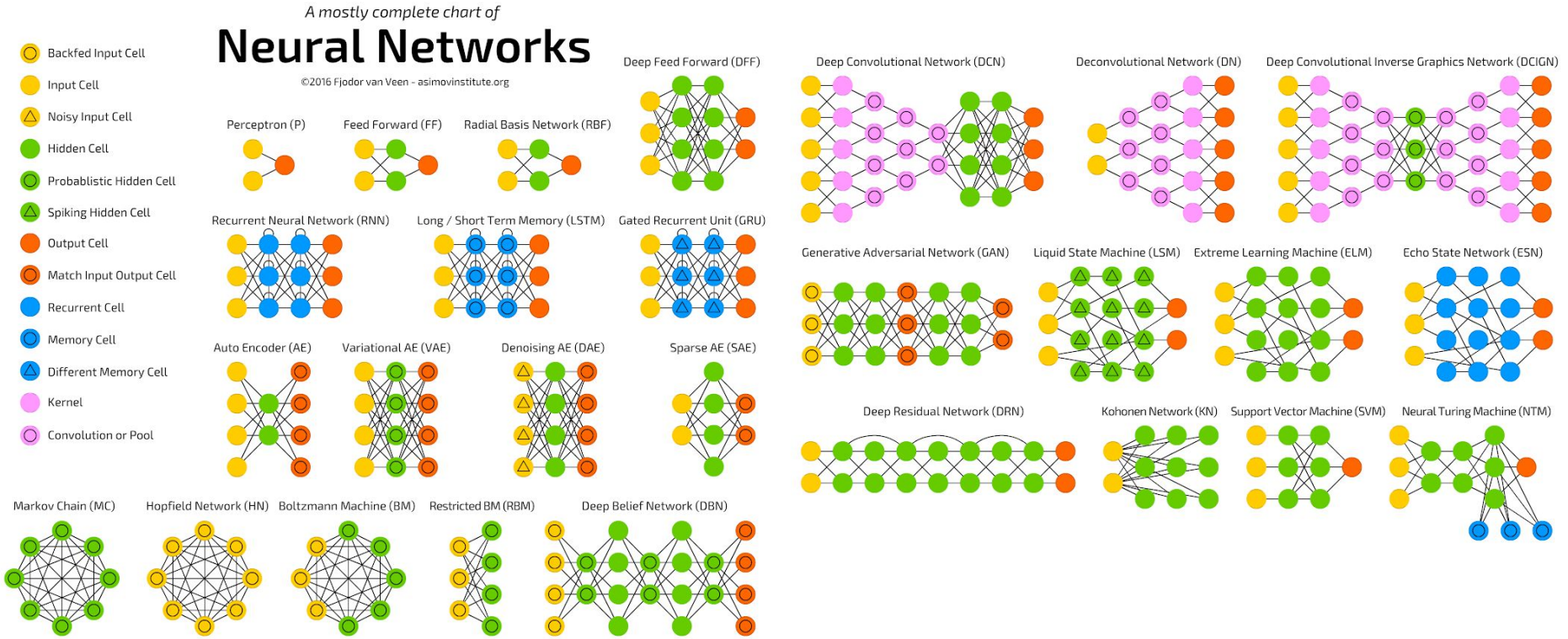
Basic Concepts

Warm up - Key Questions

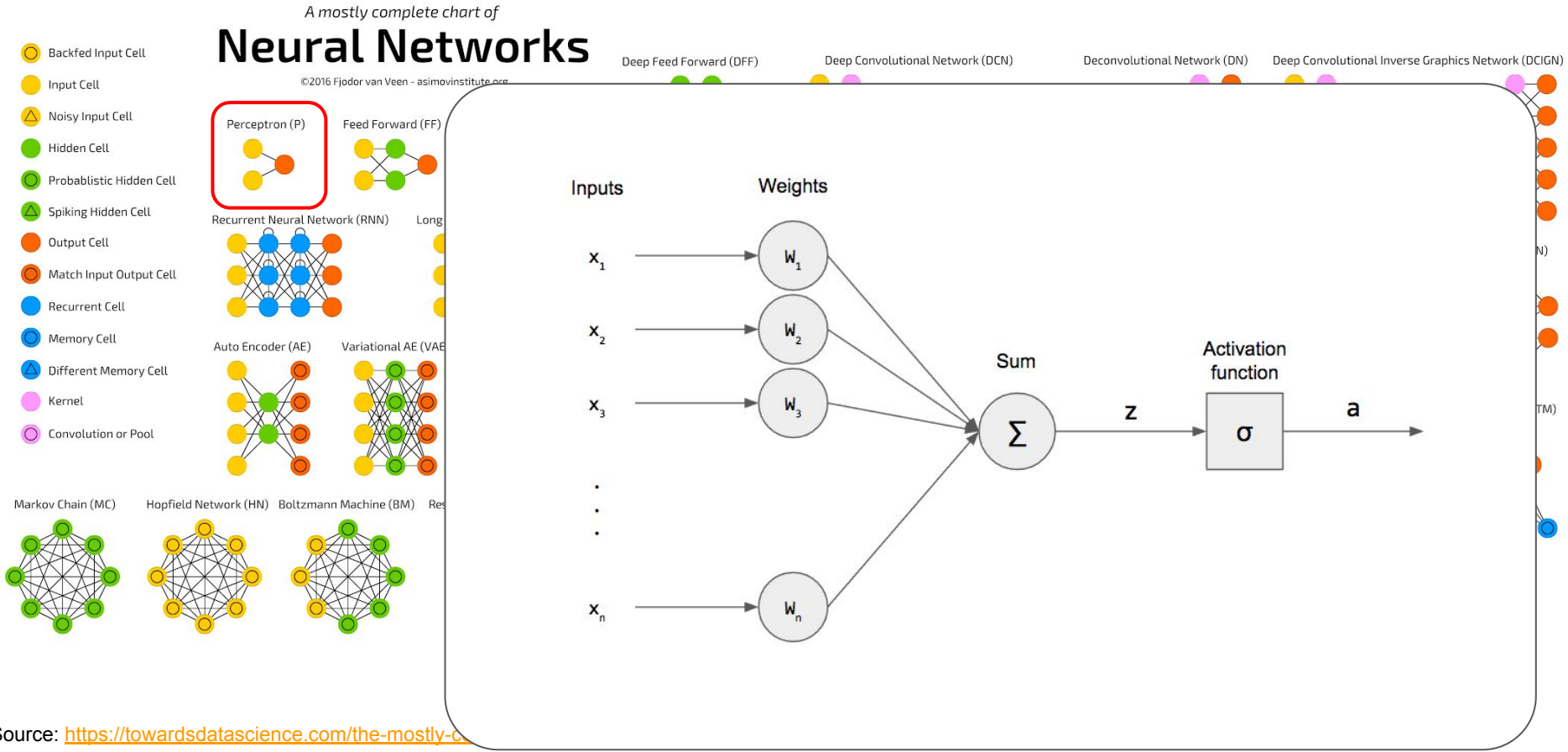
- What is a perceptron?
- What is a loss function?
- How to train a perceptron?

https://www.menti.com/alzvcmuh62tw

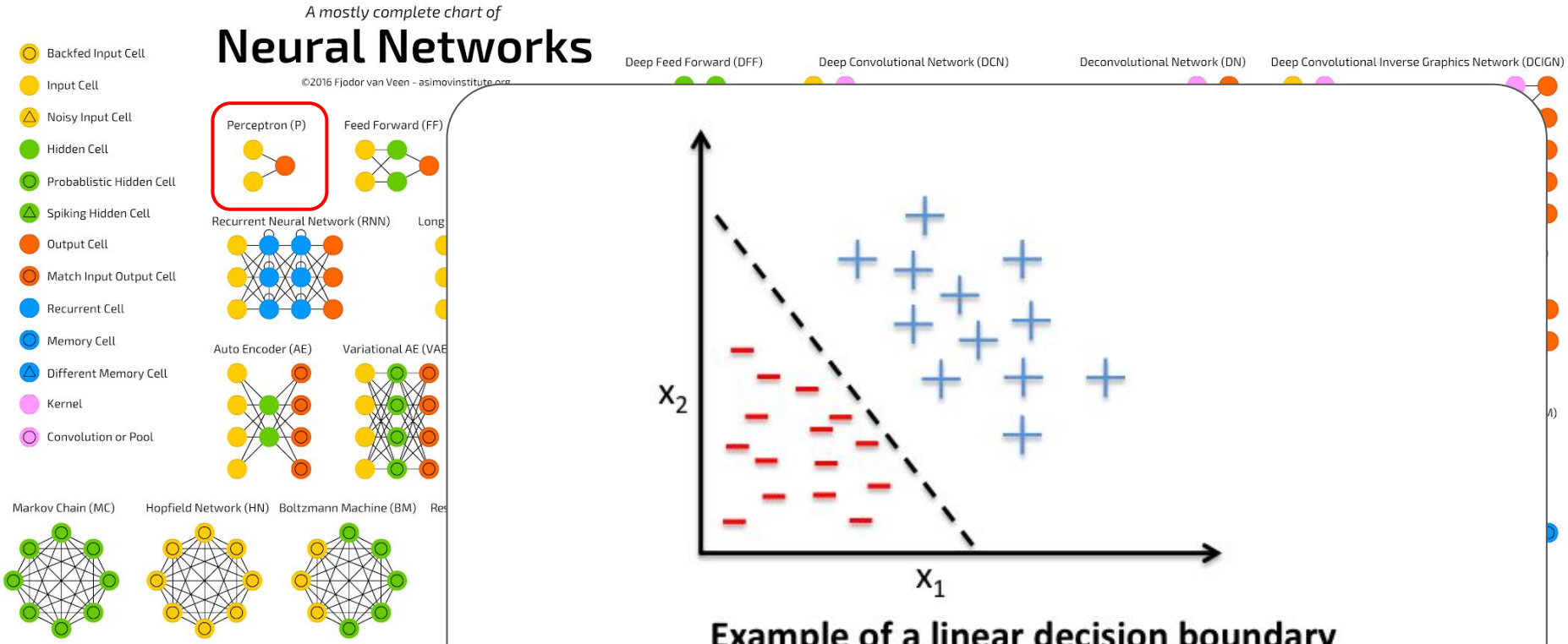
Results: https://www.mentimeter.com/app/presentation/alze25f5p8mt6foh3o1sbjuz5mk7ioby



Source: https://towardsdatascience.com/the-mostly-complete-chart-of-neural-networks-explained-3fb6f2367464



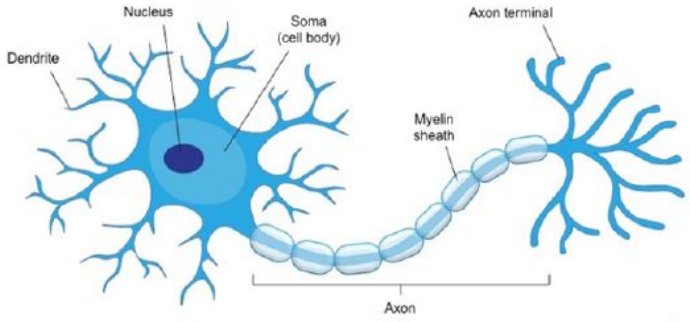
Source: https://towardsdatascience.com/the-mostly-c

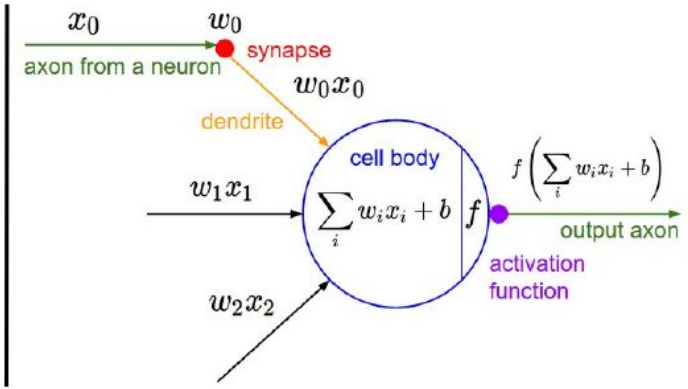


Source: https://towardsdatascience.com/the-mostly-c

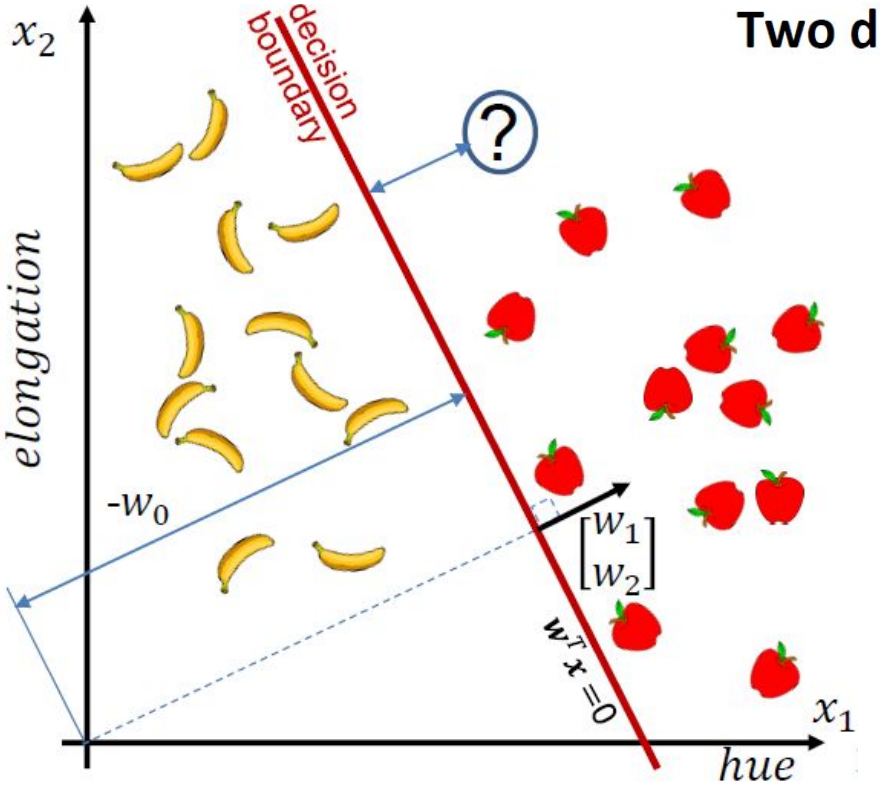
Perceptron

- Originally inspired by the human neuron. Three main parts:
  - Dendrites: Receive signals from other neurons connected to it
  - Body: Responsible for “processing” information
  - Axon: Transmits the processed signal to other neurons. Axon activation depends on the synaptic strength of the signal.





Linear Classifier



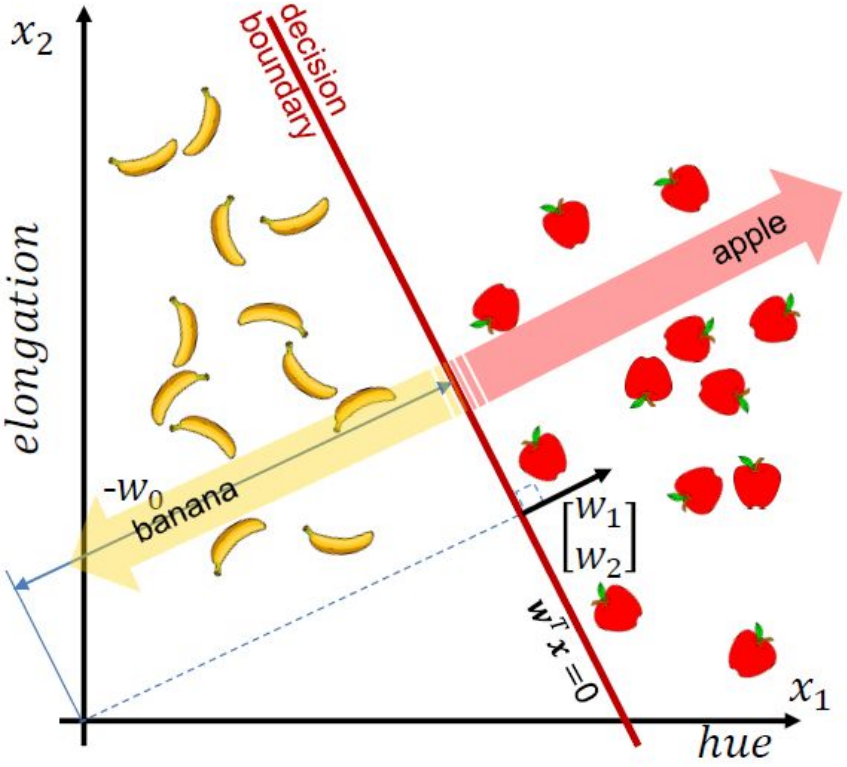
Two dimensional

$$\mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2$$

distance from the decision boundary

Adapted from Raul Feitosa (PUC-Rio)

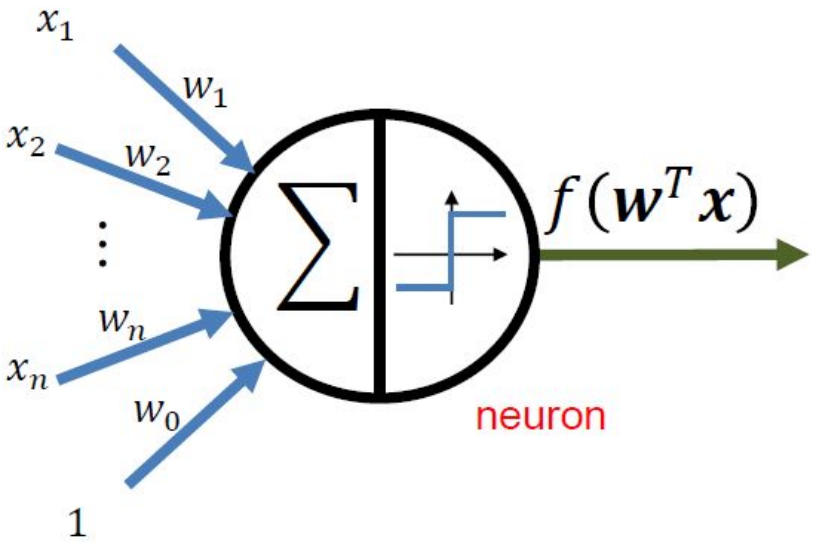
Linear Classifier



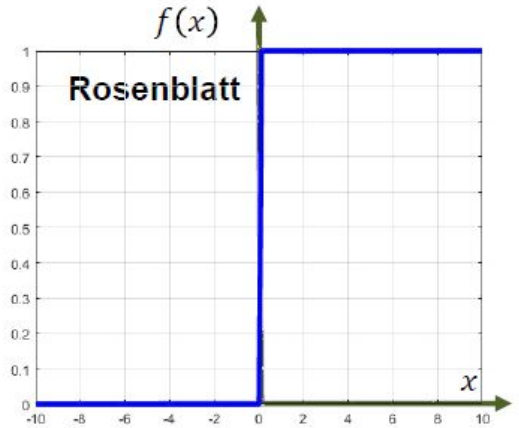
$$f(\boldsymbol{w}^T\boldsymbol{x}) = \begin{cases} 1 \; if \; \boldsymbol{w}^T\boldsymbol{x} \geq 0 \\ 0 \; otherwise \end{cases}$$

Adapted from Raul Feitosa (PUC-Rio)

Linear Classifier - Rosenblatt Perceptron (1958)

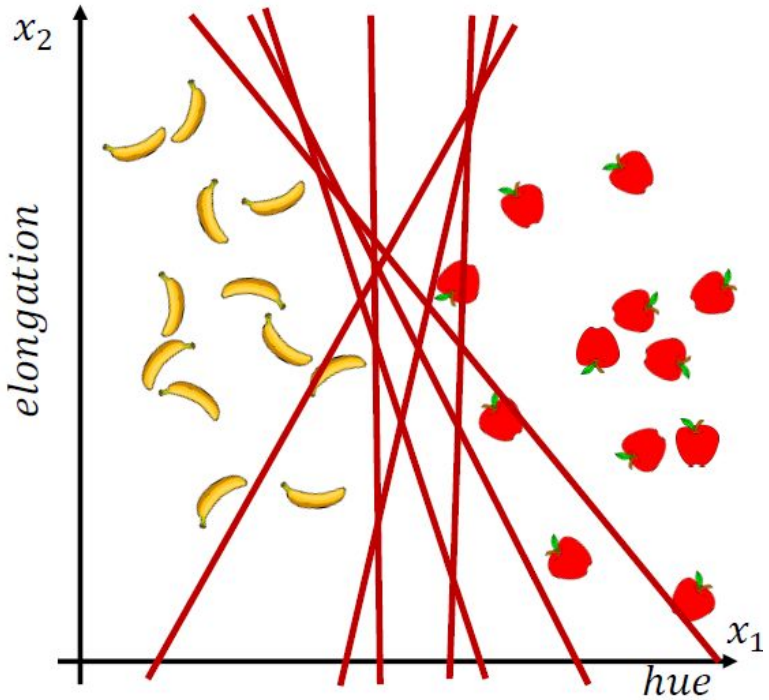


$$f(\boldsymbol{w}^T\boldsymbol{x}) = \begin{cases} 1 \; if \; \boldsymbol{w}^T\boldsymbol{x} \geq 0 \\ 0 \; otherwise \end{cases}$$



Adapted from Raul Feitosa (PUC-Rio)

Ok, but what is the best boundary?

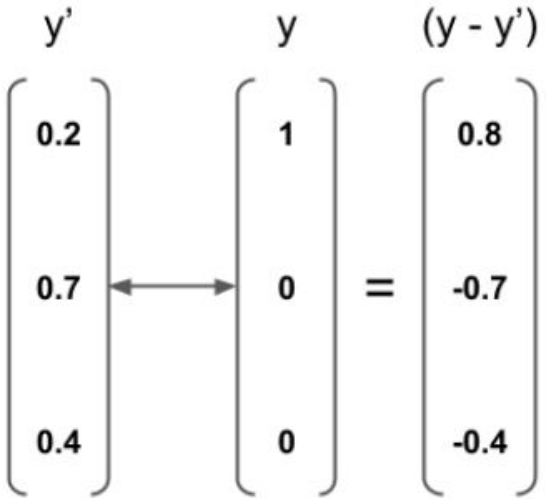


... the optimal $f(\boldsymbol{w}^T\boldsymbol{x})$?

Adapted from Raul Feitosa (PUC-Rio)

Ok, but what is the best boundary?

We need a function that tells how good is a classifier $f(\boldsymbol{W}, \boldsymbol{x}_i)$, given a set of labeled samples $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$



Adapted from Raul Feitosa (PUC-Rio)

Loss Function

We need a function that tells how good is a classifier $f(\boldsymbol{W}, \boldsymbol{x}_i)$, given a set of labeled samples $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$

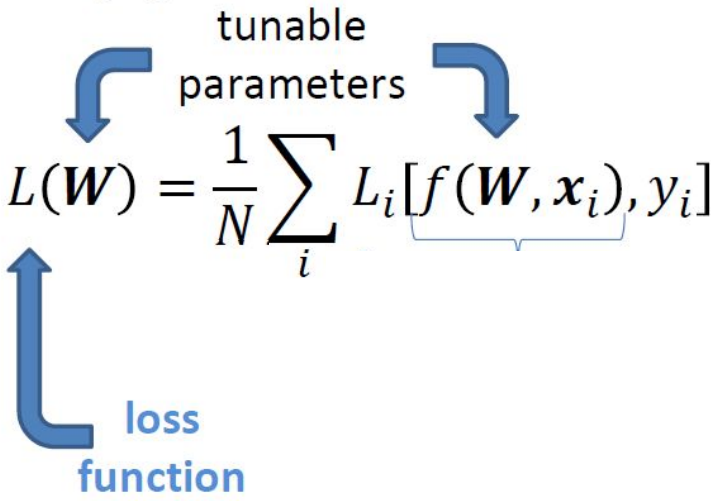
$$L(\boldsymbol{W}) = \frac{1}{N} \sum_i L_i[f(\boldsymbol{W}, \boldsymbol{x}_i), y_i]$$

loss function

Adapted from Raul Feitosa (PUC-Rio)

Loss Function

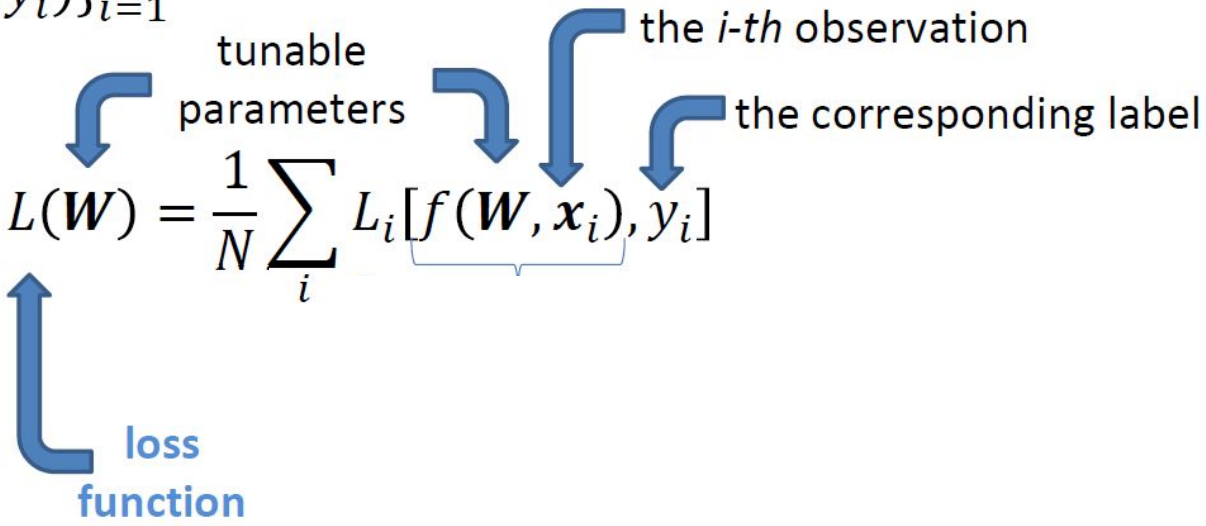
We need a function that tells how good is a classifier $f(\boldsymbol{W}, \boldsymbol{x}_i)$, given a set of labeled samples $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$



Adapted from Raul Feitosa (PUC-Rio)

Loss Function

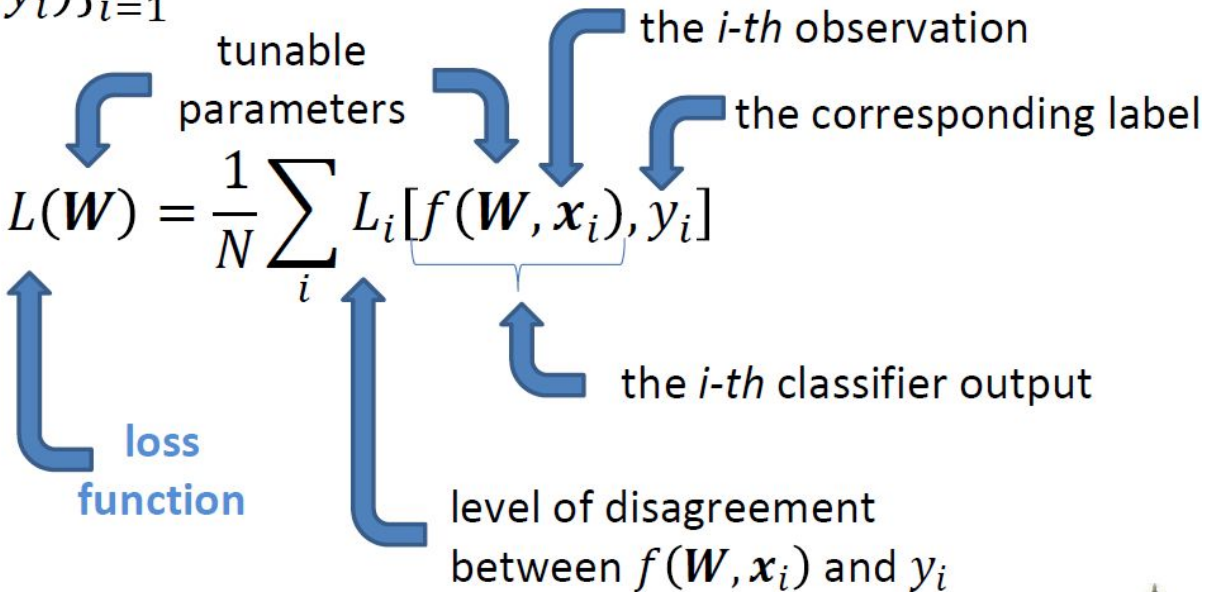
We need a function that tells how good is a classifier $f(\boldsymbol{W}, \boldsymbol{x}_i)$, given a set of labeled samples $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$



Adapted from Raul Feitosa (PUC-Rio)

Loss Function

We need a function that tells how good is a classifier $f(\boldsymbol{W}, \boldsymbol{x}_i)$, given a set of labeled samples $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{N}$

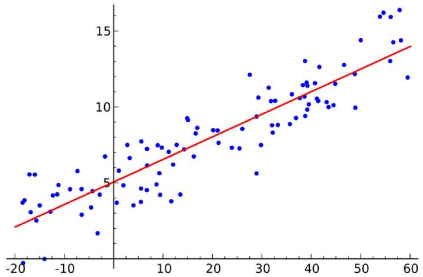


Adapted from Raul Feitosa (PUC-Rio)

Some Common Loss Functions

- Cross Entropy Loss - Classification
- Dice Loss - Segmentation
- Focal Loss - Segmentation and Detection
- L1 Loss - Regression/Reconstruction
- Kullback-Leibler Divergence Loss - Statistical Distributions
- **MSE Loss - Regression/Reconstruction**

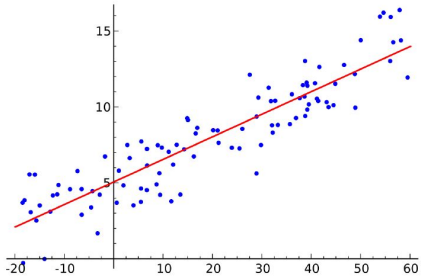
$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$



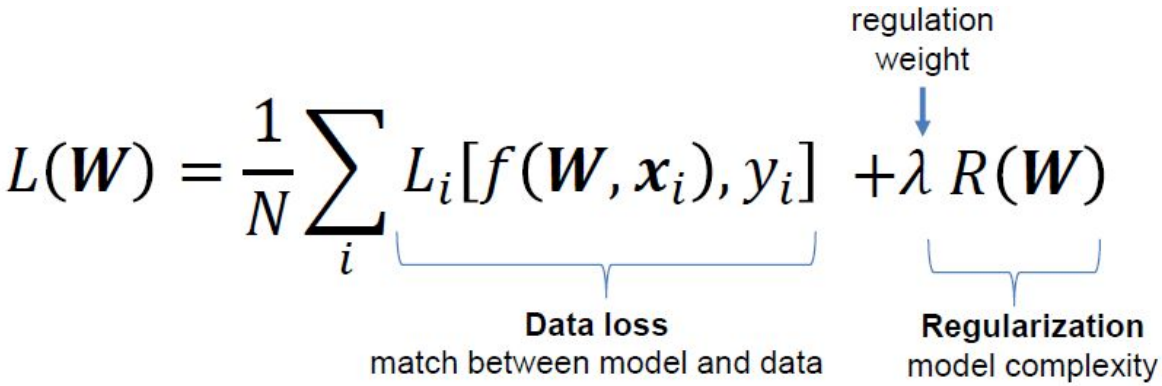
Some Common Loss Functions

- Cross Entropy Loss - Classification ← Next week with ConvNets!
- Dice Loss - Segmentation
- Focal Loss - Segmentation and Detection
- L1 Loss - Regression/Reconstruction
- Kullback-Leibler Divergence Loss - Statistical Distributions
- **MSE Loss - Regression/Reconstruction**

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$



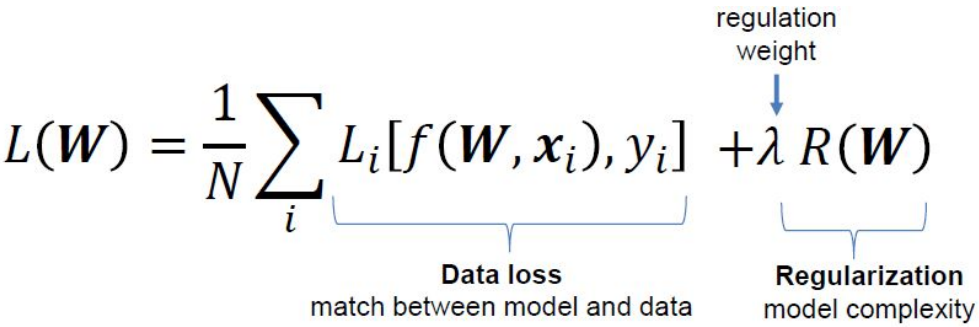
Loss Function & Regularization



Adapted from Raul Feitosa (PUC-Rio)

Model Complexy & Regularization

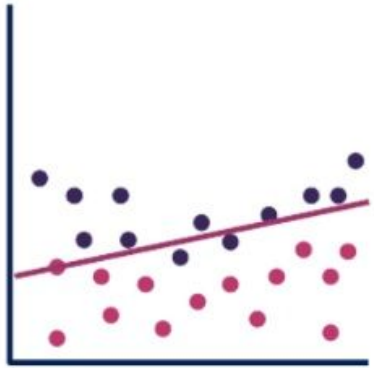
Regularization in machine learning is a technique used to prevent overfitting and improve the generalization ability of machine learning models.

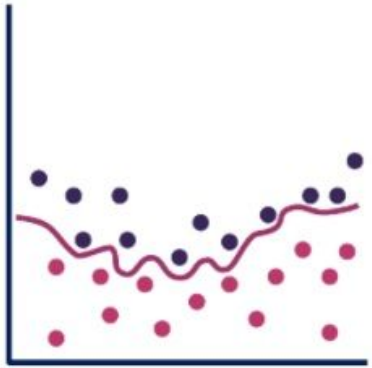


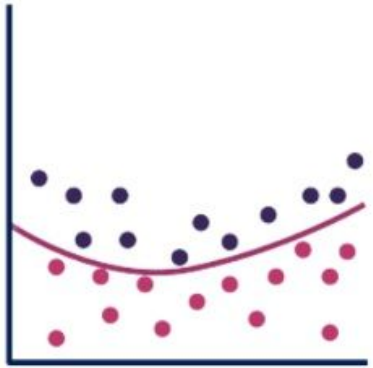
Regularization methods introduce additional constraints or penalties to the learning process, encouraging the model to learn simpler patterns that are more likely to generalize well.

Model Complexity

What is the better model?



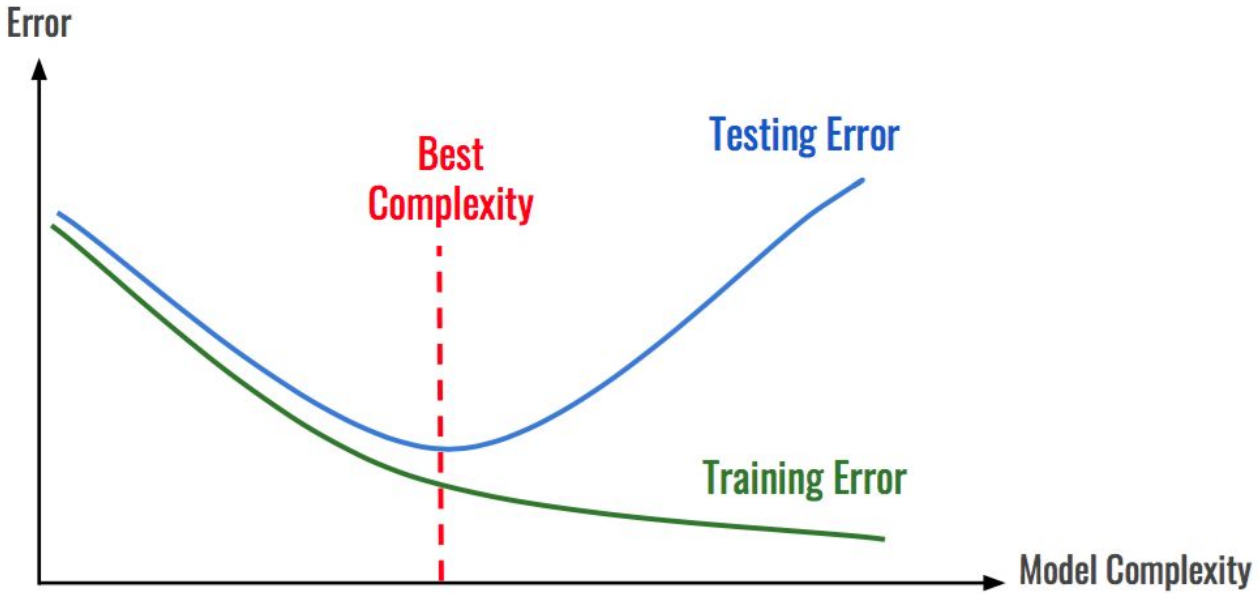




Model Complexity

What is the better model?

Model Complexity



Model Complexy & Regularization

Common regularization techniques:

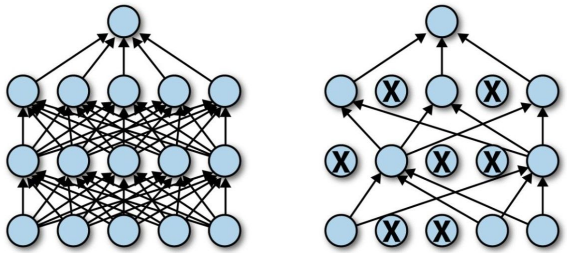
- L1
  - Adds a penalty term proportional to the absolute value of the model's coefficients to the loss function.
  - Encourages sparsity in the model by driving some coefficients to exactly zero, effectively performing feature selection
- L2
  - Adds a penalty term proportional to the square of the model's coefficients to the loss function
- Dropout
  - Randomly sets a fraction of neurons' outputs to zero, effectively "dropping out" those neurons from the network for that iteration
- Early Stopping
  - Monitors the model's performance on a validation dataset during training. Training is stopped when the validation performance starts to degrade, preventing the model from overfitting to the training data.

L1 Regularization

$$\text{Modified loss function} = \text{Loss function} + \lambda \sum_{i=1}^{n} |W_i|$$

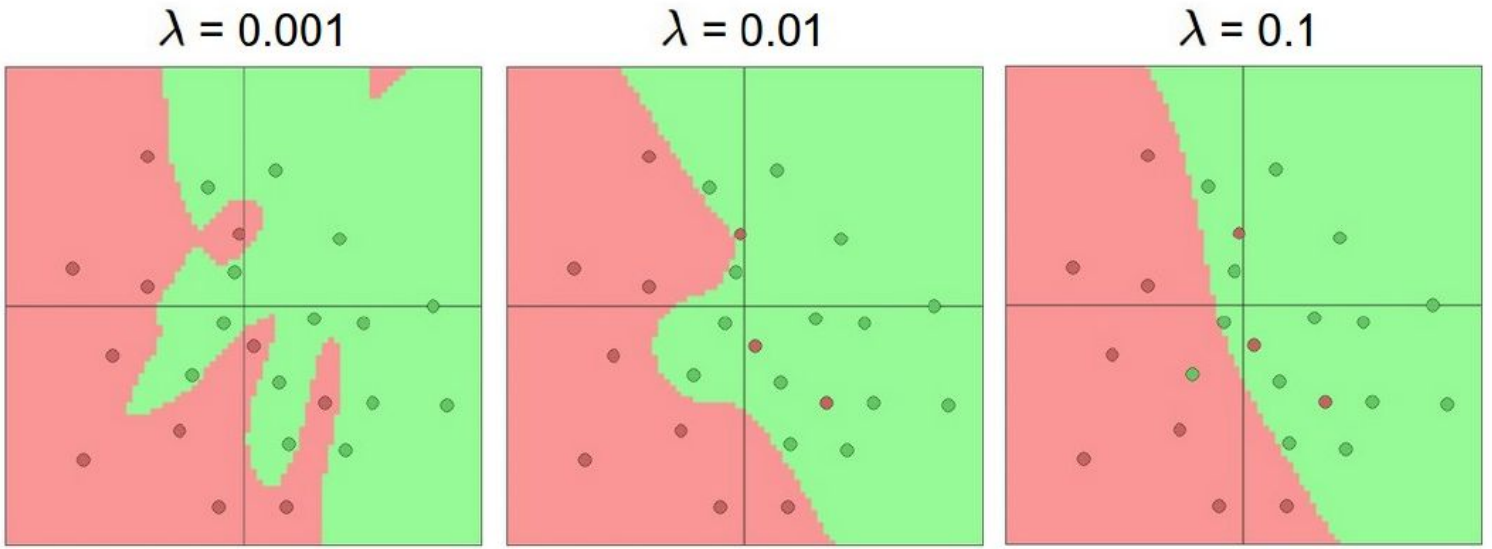
L2 Regularization

$$\text{Modified loss function} = \text{Loss function} + \lambda \sum_{i=1}^{n} W_i^2$$



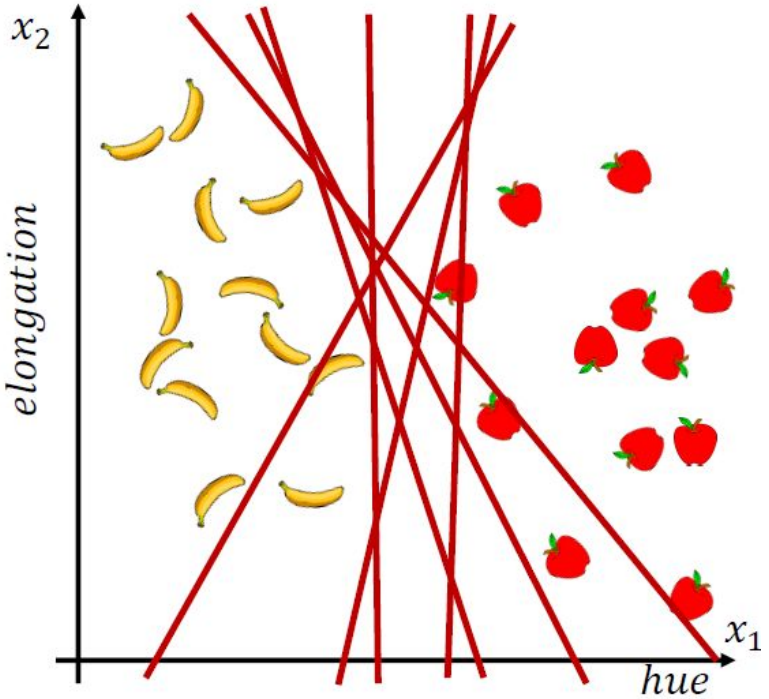
Model Complexy & Regularization

Different regularization values (L2):



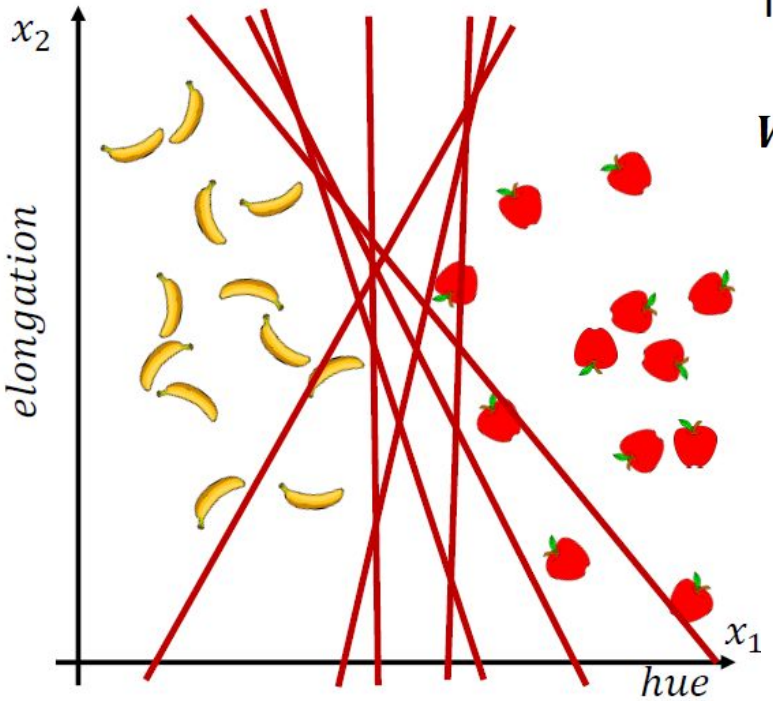
Training a Perceptron

What is the best boundary?



Adapted from Raul Feitosa (PUC-Rio)

What is the best boundary?



The best $\boldsymbol{W}$ ($\boldsymbol{W}_{opt}$) minimizes the Loss:

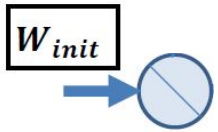
$$\boldsymbol{W}_{opt} = \underset{\boldsymbol{W}}{\operatorname{argmin}} \left\{ \frac{1}{N} \sum_i L_i[f(\boldsymbol{W}, \boldsymbol{x}_i), y_i] + \lambda\, R(\boldsymbol{W}) \right\}$$

Adapted from Raul Feitosa (PUC-Rio)

Looking for the best *W*

The best $\boldsymbol{W}$ ($\boldsymbol{W}_{opt}$) minimizes the Loss:

$$\boldsymbol{W}_{opt} = \underset{\boldsymbol{W}}{\operatorname{argmin}} \left\{ \frac{1}{N} \sum_i L_i[f(\boldsymbol{W}, \boldsymbol{x}_i), y_i] + \lambda \, R(\boldsymbol{W}) \right\}$$

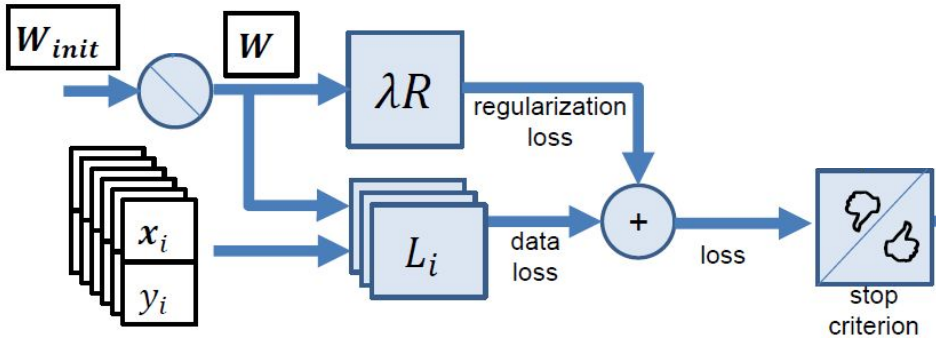


Adapted from Raul Feitosa (PUC-Rio)

Looking for the best *W*

The best $\boldsymbol{W}$ ($\boldsymbol{W}_{opt}$) minimizes the Loss:

$$\boldsymbol{W}_{opt} = \underset{\boldsymbol{W}}{\operatorname{argmin}} \left\{ \frac{1}{N} \sum_i L_i[f(\boldsymbol{W}, \boldsymbol{x}_i), y_i] + \lambda\, R(\boldsymbol{W}) \right\}$$

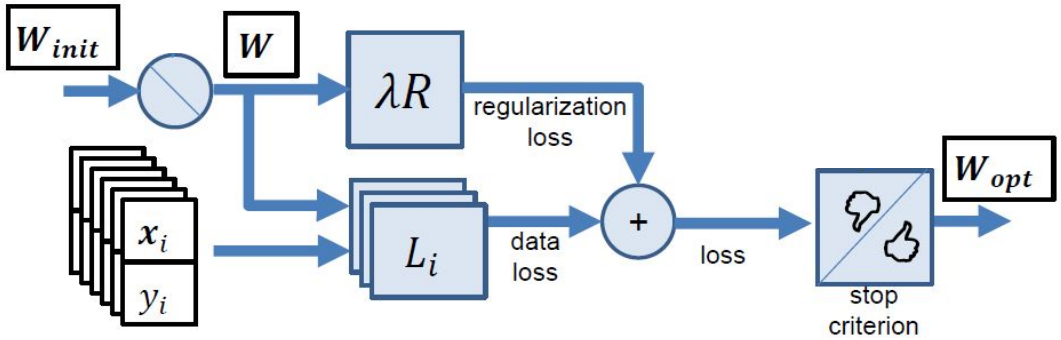


Adapted from Raul Feitosa (PUC-Rio)

Looking for the best *W*

The best $\boldsymbol{W}$ ($\boldsymbol{W}_{opt}$) minimizes the Loss:

$$\boldsymbol{W}_{opt} = \underset{\boldsymbol{W}}{\operatorname{argmin}} \left\{ \frac{1}{N} \sum_i L_i[f(\boldsymbol{W}, \boldsymbol{x}_i), y_i] + \lambda\, R(\boldsymbol{W}) \right\}$$

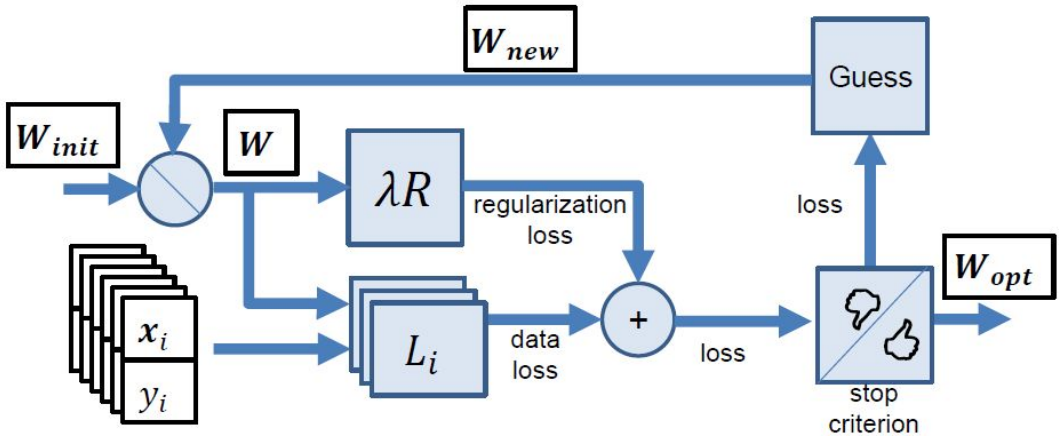


Adapted from Raul Feitosa (PUC-Rio)

Looking for the best *W*

The best $\boldsymbol{W}$ ($\boldsymbol{W}_{opt}$) minimizes the Loss:

$$\boldsymbol{W}_{opt} = \underset{\boldsymbol{W}}{\operatorname{argmin}} \left\{ \frac{1}{N} \sum_i L_i[f(\boldsymbol{W}, \boldsymbol{x}_i), y_i] + \lambda\, R(\boldsymbol{W}) \right\}$$

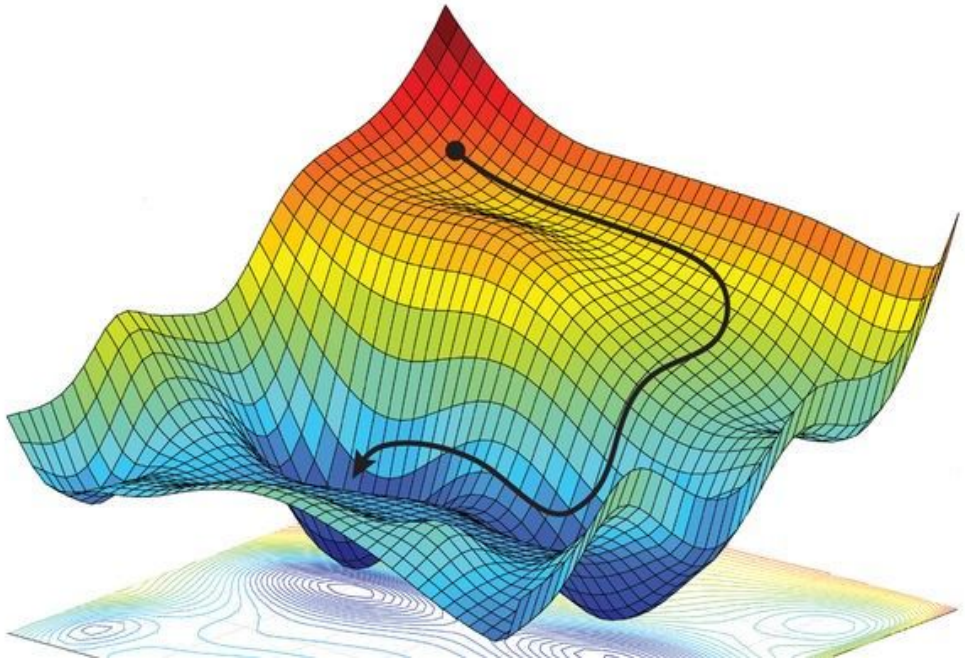


Adapted from Raul Feitosa (PUC-Rio)

Optimization

Image credits: Fei-Fei Li & Andrej Karpathy & Justin Johnson

Gradient Based Optimization



Recommended videos:
https://youtu.be/qg4PchTECck
https://youtu.be/uJryes5Vk1o

More details on how to compute the gradient:
Jason Brownlee: https://machinelearningmastery.com/gradient-in-machine-learning/

Image credits: Kamil Krzyk

Stochastic Gradient Descent (SGD)

Move towards the deepest valley

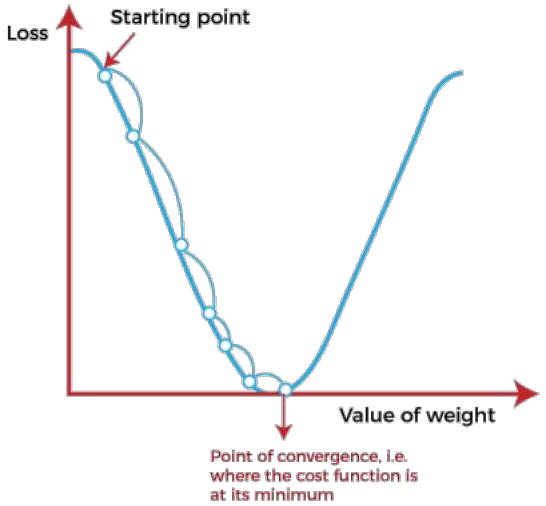
$$\boldsymbol{W} \leftarrow \boldsymbol{W} - \alpha \nabla_{\boldsymbol{W}} L(\boldsymbol{W})$$

learning rate

gradient of Loss

until

- Number of iterations is reached, or
- $\nabla_{\boldsymbol{W}} L(\boldsymbol{W})$ is small enough, or
- ...



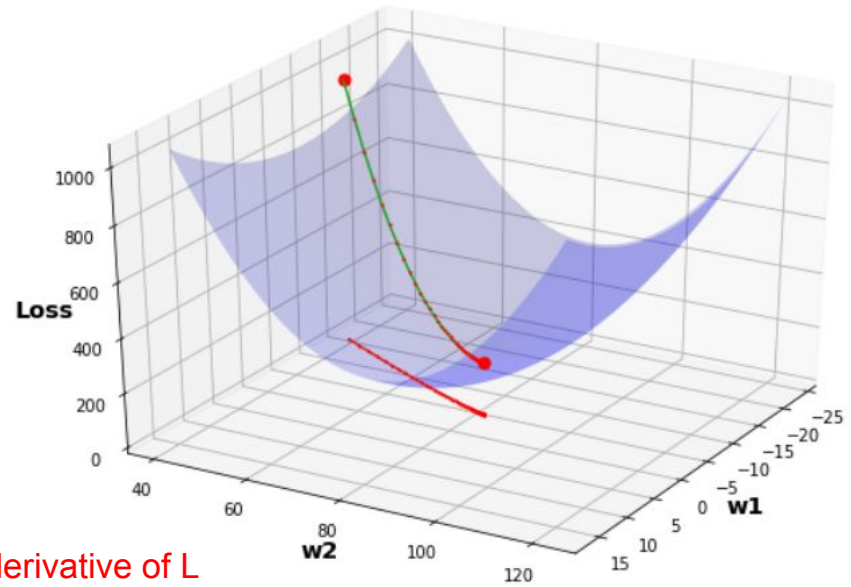
Adapted from Raul Feitosa (PUC-Rio)

Stochastic Gradient Descent (SGD)

- The direction is given by the **derivative** of cost with respect to weight.
- For multiple dimensions the vector of partial derivatives is called a **gradient**
- **Gradient descent:**

$$W_{ij}^{(L)} = W_{ij}^{(L)} - \alpha \frac{\partial \mathcal{L}(W,b)}{\partial W_{ij}^{(L)}}$$

$$b_{ij}^{(L)} = b_{ij}^{(L)} - \alpha \frac{\partial \mathcal{L}(W,b)}{\partial b_{ij}^{(L)}}$$



What is the derivative of L with respect to $W^{(L)}$?

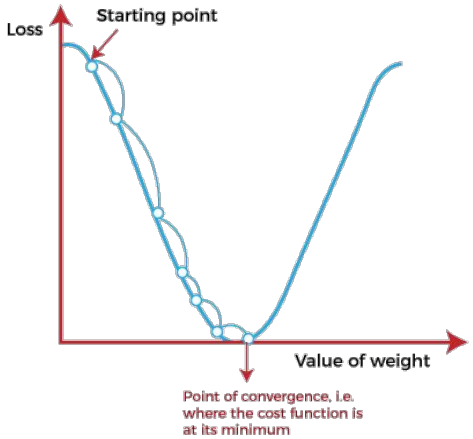
We can change the weights and compute how the loss function was impacted by the change.

Stochastic Gradient Descent (SGD)

Often large training sets are required to tune a classifier. Computing the gradient over all samples

$$\nabla_{\boldsymbol{W}} L(\boldsymbol{W}) = \frac{1}{N} \sum_{i} \nabla_{\boldsymbol{W}} L_i[f(\boldsymbol{W}, \boldsymbol{x}_i), y_i] + \lambda \nabla_{\boldsymbol{W}} R(\boldsymbol{W})$$

might be extremely expensive $(O(N))$!



Adapted from Raul Feitosa (PUC-Rio)

SGD with Minibatches

Sample uniformly **minibatches** of samples $\mathbb{B} = \{x^{(1)}, \ldots, x^{(m)}\}$, for a "small" m.

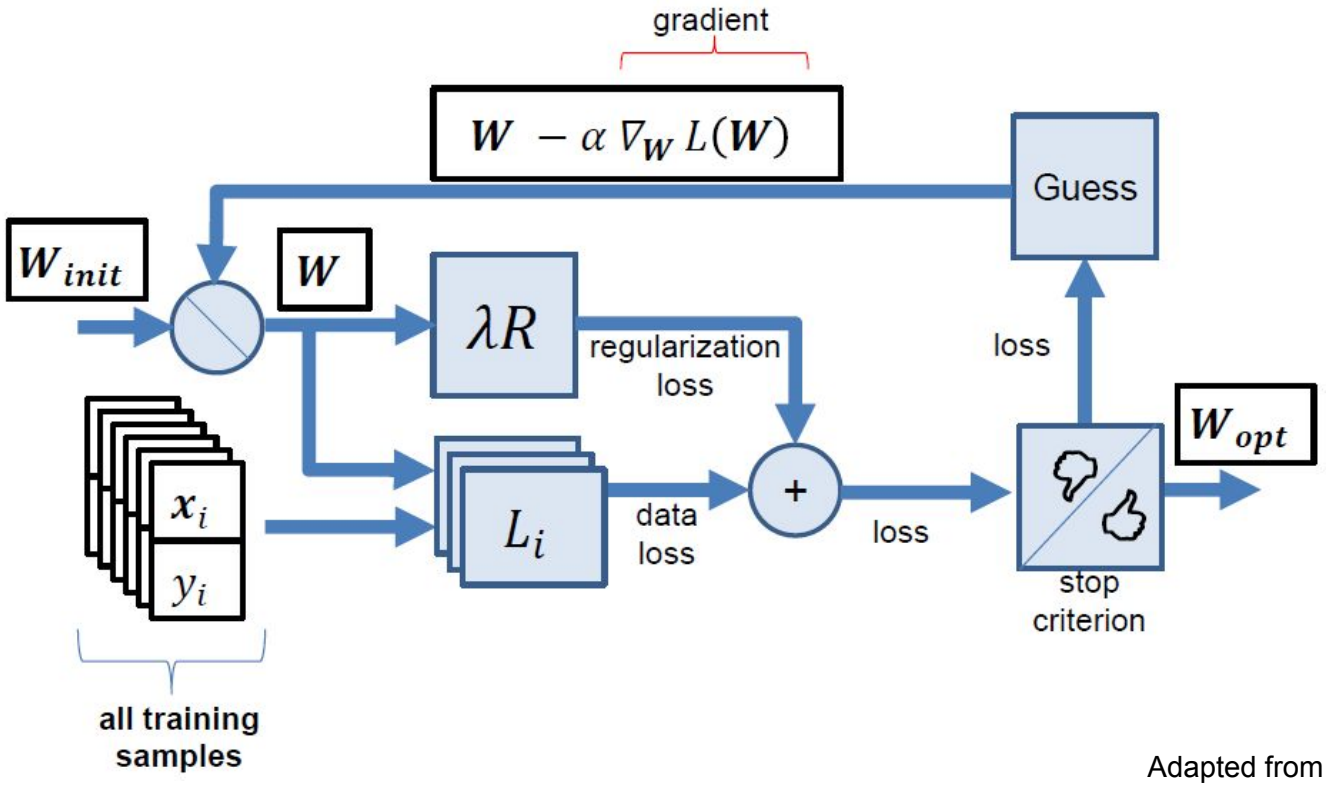
At each step estimate the gradient upon a minibatch

$$\boldsymbol{g}_{\mathbb{B}} = \frac{1}{m} \sum_{\boldsymbol{x}_i \in \mathbb{B}} \nabla_{\boldsymbol{W}} L_i[f(\boldsymbol{W}, \boldsymbol{x}_i), y_i] + \lambda \nabla_{\boldsymbol{W}} R(\boldsymbol{W})$$

and updated $\boldsymbol{W}$ as $\boldsymbol{W} \leftarrow \boldsymbol{W} - \alpha \boldsymbol{g}_{\mathbb{B}}$

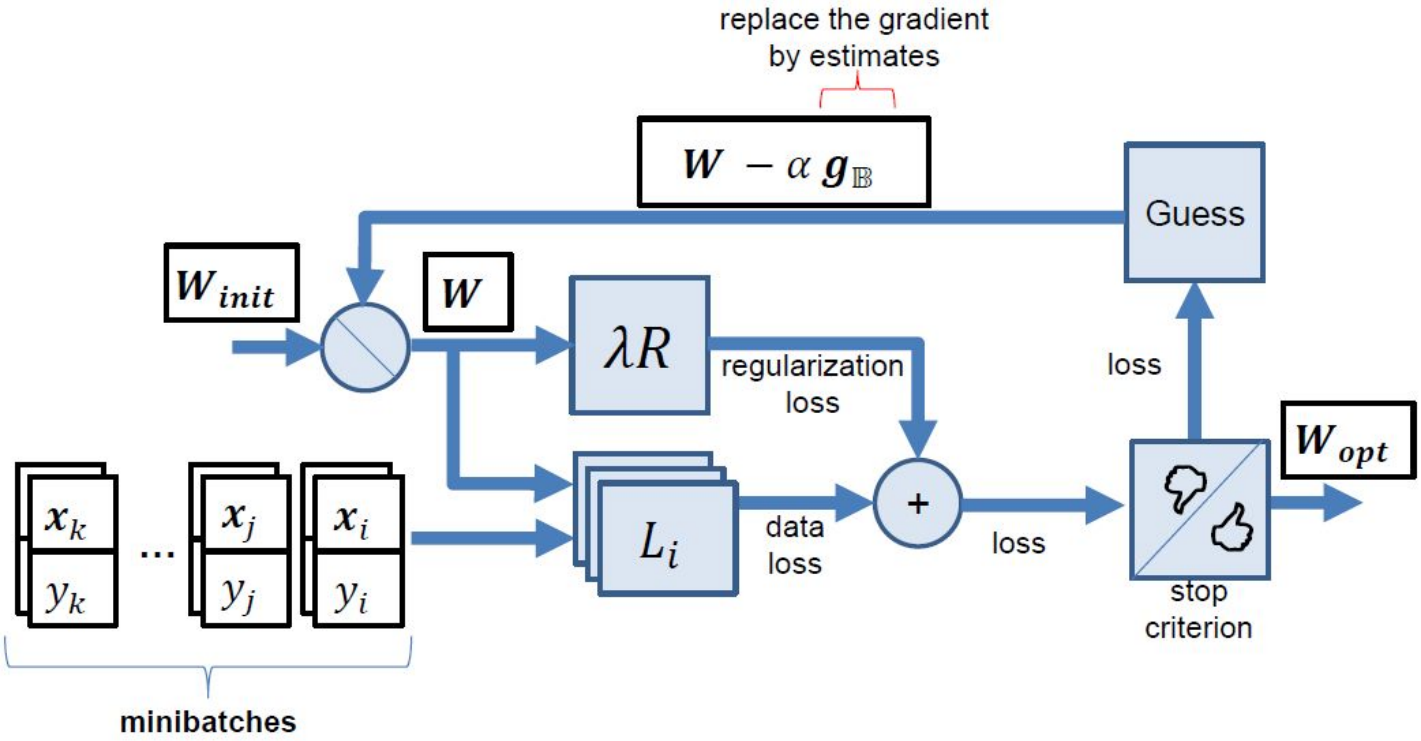
Adapted from Raul Feitosa (PUC-Rio)

Original SGD



Adapted from Raul Feitosa (PUC-Rio)

SGD with Mini Batches



Adapted from Raul Feitosa (PUC-Rio)

Gradient Based Optimization

- Batch gradient descent
- Stochastic gradient descent
- Mini-batch gradient descent
- Momentum
- Nesterov accelerated gradient
- Adagrad
- Adadelta
- RMSprop
- Adam
- AdaMax
- Nadam
- AMSGrad

Key Concepts

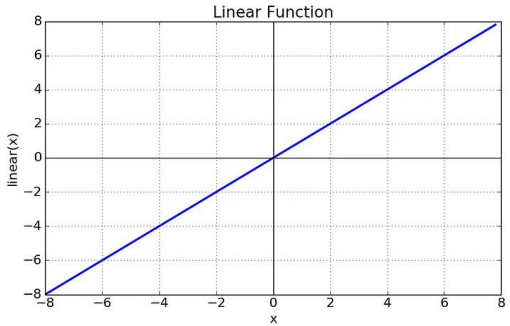
- **Activation functions**

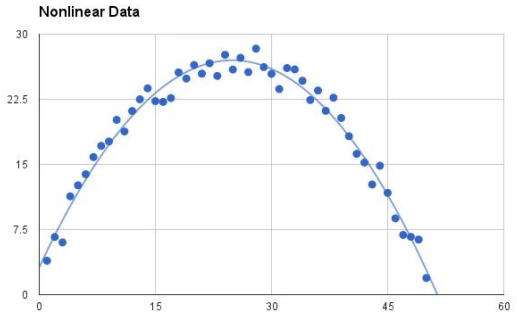
Activation Functions

A function that you use to get or change the output of a perceptron (a.k.a. Transfer Function).

Two types:

- Linear Activation Function
- Non-linear Activation Functions





Activation Functions

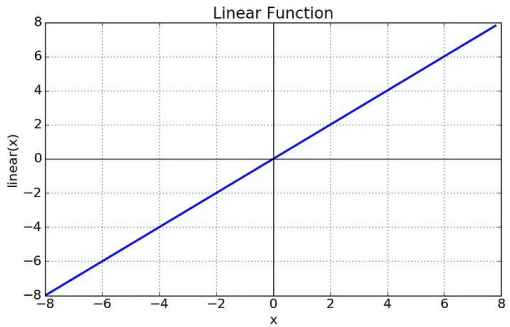
A function that you use to get or change the output of a perceptron (a.k.a. Transfer Function).

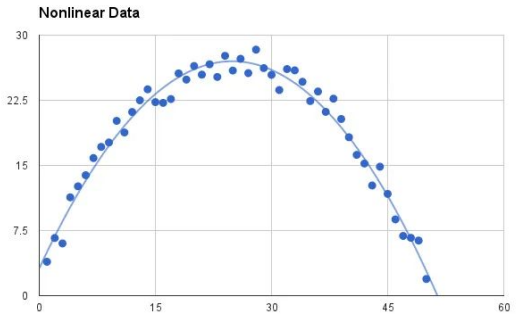
Two types:

- Linear Activation Function
- Non-linear Activation Functions

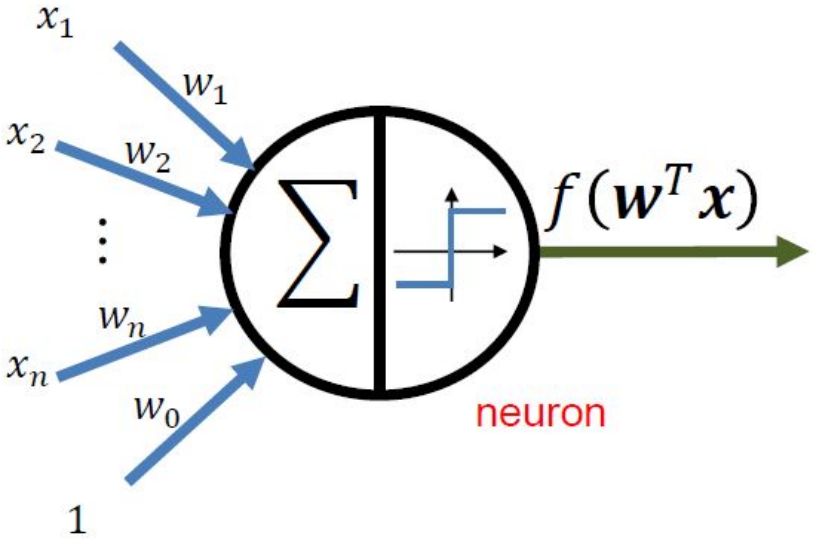
Remember:

- **Training depends on derivatives computation (gradient)**

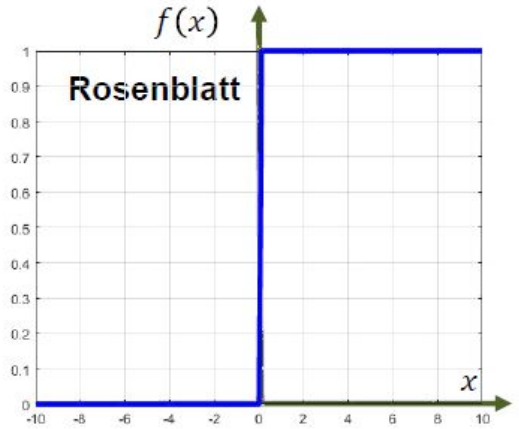




Rosenblatt Perceptron

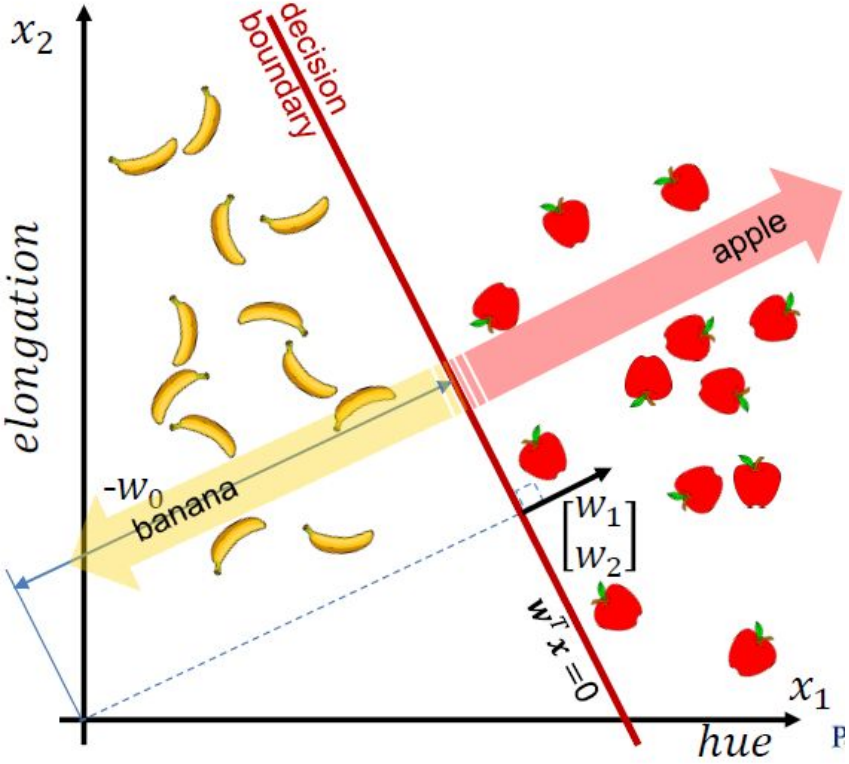


$$f(\boldsymbol{w}^T\boldsymbol{x}) = \begin{cases} 1 \; if \; \boldsymbol{w}^T\boldsymbol{x} \geq 0 \\ 0 \; otherwise \end{cases}$$



Adapted from Raul Feitosa (PUC-Rio)

Rosenblatt Perceptron



$$f(\boldsymbol{w}^T \boldsymbol{x}) = \begin{cases} 1 \; if \; \boldsymbol{w}^T \boldsymbol{x} \geq 0 \\ 0 \; otherwise \end{cases}$$

Adapted from Raul Feitosa (PUC-Rio)

Logistic Function (Sigmoid Activation Function)

$$f(\boldsymbol{w}^T\boldsymbol{x}) = \frac{1}{1+e^{-\boldsymbol{w}^T\boldsymbol{x}}} = \sigma(\boldsymbol{w}^T\boldsymbol{x})$$

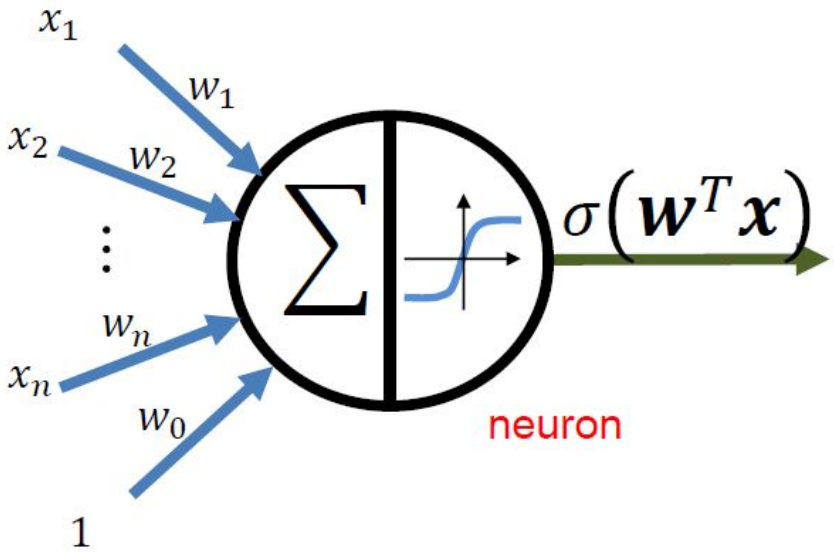
Remember:

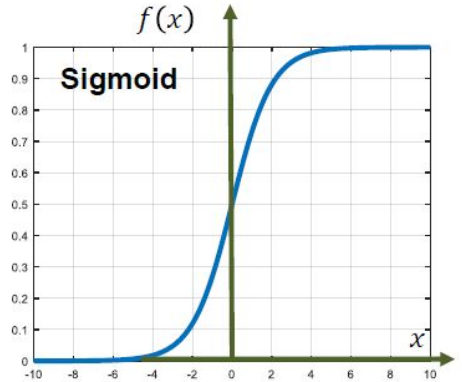
- Training depends on derivatives computation (gradient)

Sigmoid function has amazing derivative properties!

Logistic Function (Sigmoid Activation Function) - Softmax

$$f(\boldsymbol{w}^T\boldsymbol{x}) = \frac{1}{1+e^{-\boldsymbol{w}^T\boldsymbol{x}}} = \sigma(\boldsymbol{w}^T\boldsymbol{x})$$



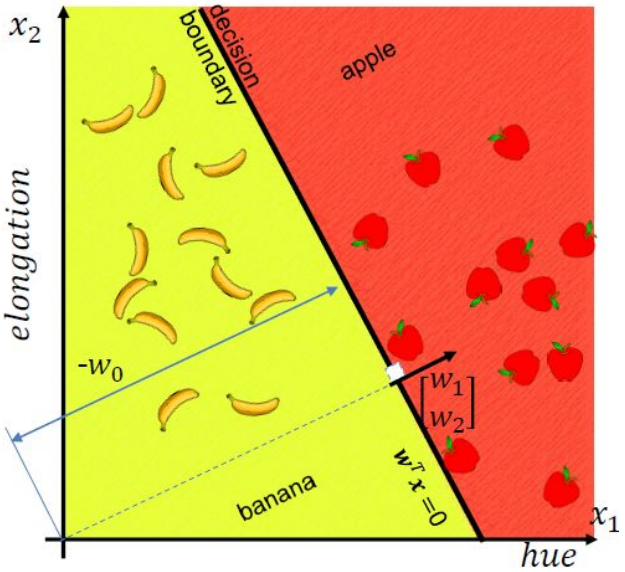


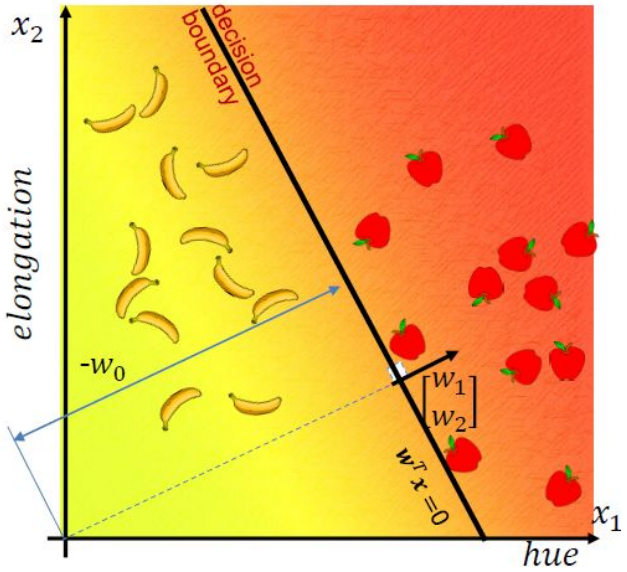
Logistic Function (Sigmoid Activation Function)

Two dimensional

$$\boldsymbol{w}^T\boldsymbol{x} = w_0 + w_1 x_1 + w_2 x_2$$

distance from the decision boundary





Most common Activation Functions

- Sigmoid (Logistic)
- Hyperbolic Tangent (Tanh)
- Rectified Linear Unit (ReLU)
- Leaky ReLU.
- Parametric Leaky ReLU (PReLU)
- Exponential Linear Units (ELU)
- Scaled Exponential Linear Unit (SELU)

Most common Activation Functions

- Sigmoid (Logistic)
- Hyperbolic Tangent (Tanh)
- Rectified Linear Unit (ReLU)
- Leaky ReLU.
- Parametric Leaky ReLU (PReLU)
- Exponential Linear Units (ELU)
- Scaled Exponential Linear Unit (SELU)

It is typically used as an activation function in the **last layer of Networks for classification**.
It is normalized to represent a probability distribution.

Most common Activation Functions

- Sigmoid (Logistic)
- Hyperbolic Tangent (Tanh)
- Rectified Linear Unit (ReLU)
- Leaky ReLU.
- Parametric Leaky ReLU (PReLU)
- Exponential Linear Units (ELU)
- Scaled Exponential Linear Unit (SELU)

We will talk more about this in near future :-)

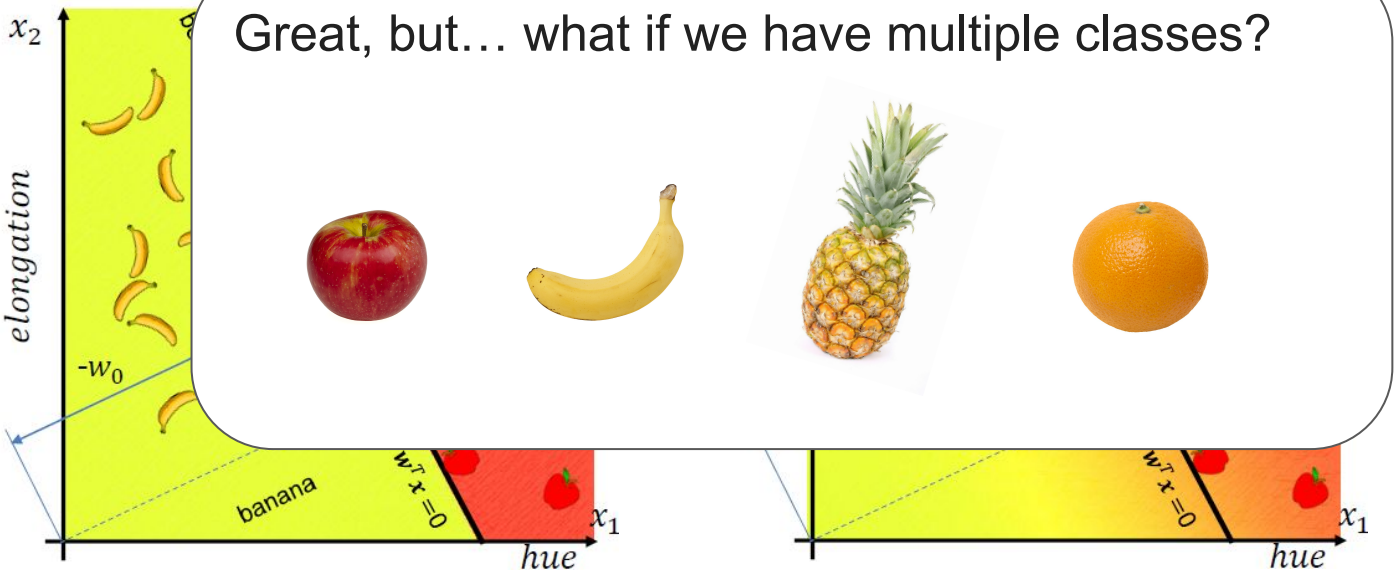
It is typically used in hidden layers for ConvNets.

Logistic Function (Sigmoid Activation Function)

Two dimensional

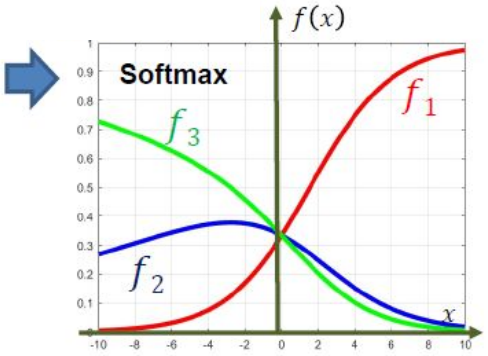
$$\boldsymbol{w}^T\boldsymbol{x} = w_0 + w_1x_1 + w_2x_2$$ ← distance from the decision boundary

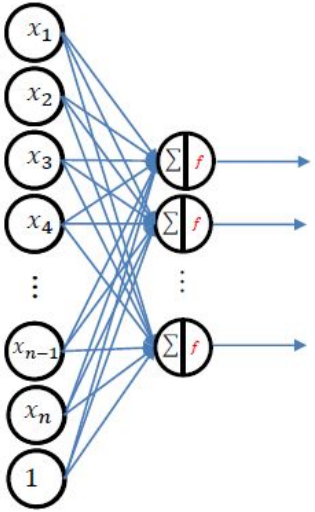
Great, but… what if we have multiple classes?

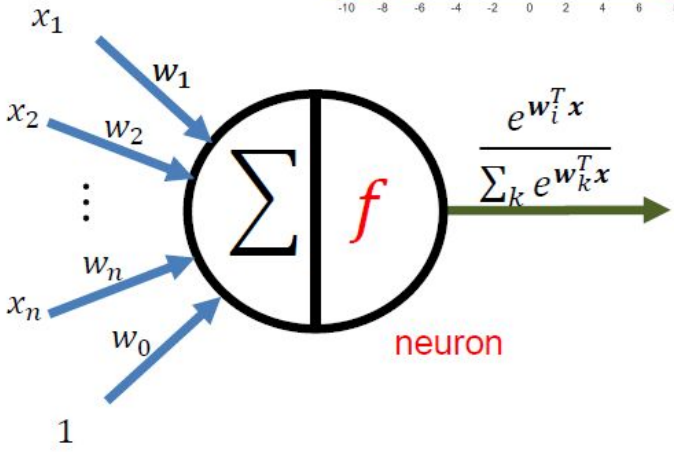


Multinomial Logistic Regression (Softmax)

$$f(\boldsymbol{w}_i^T \boldsymbol{x}) = \frac{e^{\boldsymbol{w}_i^T \boldsymbol{x}}}{\sum_k e^{\boldsymbol{w}_k^T \boldsymbol{x}}}$$







Deep Neural Networks

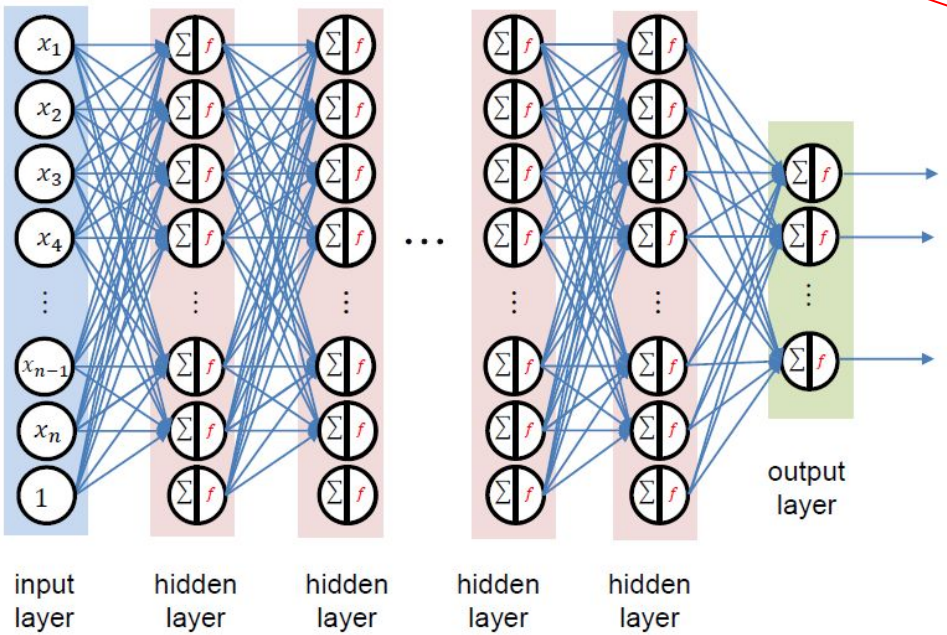
Key Questions

What is a multilayer perceptron?

How to train a multilayer perceptron?

What is a multilayer perceptron?

$$f(\boldsymbol{W}^T \boldsymbol{x}) = f_d\left(\boldsymbol{W}_d^T f_{d-1}\left(\boldsymbol{W}_{d-1}^T f_{d-2}\left(\ldots \boldsymbol{W}_2^T f_1(\boldsymbol{W}_1^T \boldsymbol{x})\right)\right)\right)$$

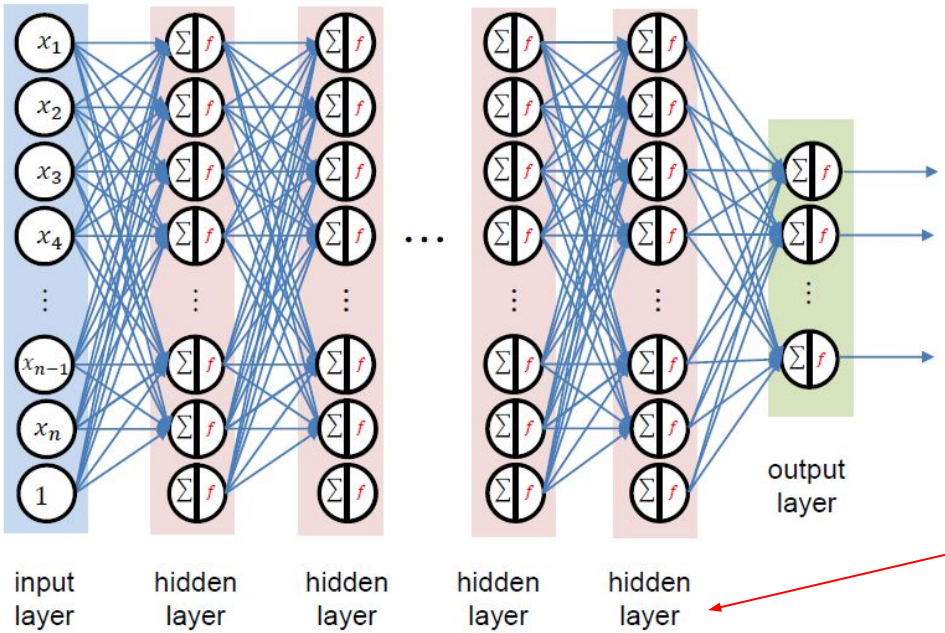


A multilayer architecture is a composite function

Adapted from Raul Feitosa (PUC-Rio)

What is a multilayer perceptron?

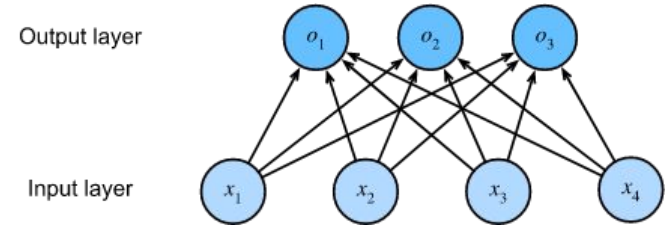
$$f(\boldsymbol{W}^T\boldsymbol{x}) = f_d\left(\boldsymbol{W}_d^T f_{d-1}\left(\boldsymbol{W}_{d-1}^T f_{d-2}\left(\ldots \boldsymbol{W}_2^T f_1(\boldsymbol{W}_1^T\boldsymbol{x})\right)\right)\right)$$



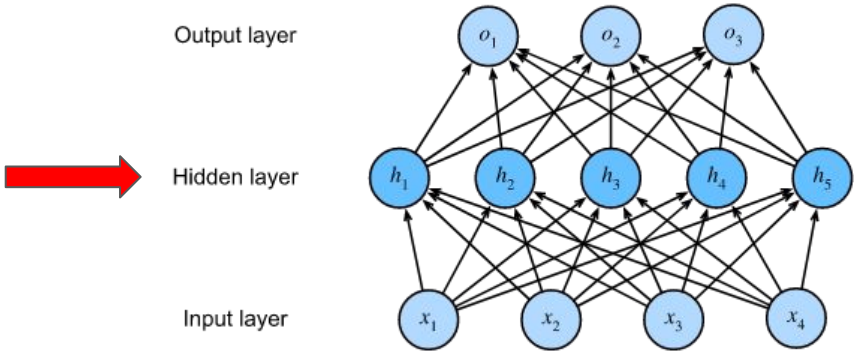
In addition to the input and output layers, we also have the hidden layers.

Adapted from Raul Feitosa (PUC-Rio)

Single Perceptron X MLP



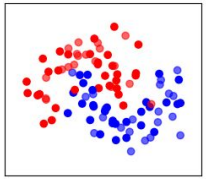
Perceptron

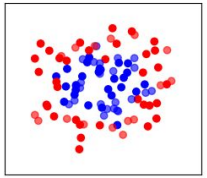


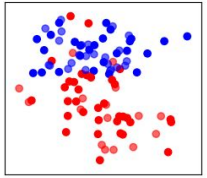
MLP

Source: Dive into Deep Learning - https://d2l.ai/

Single Perceptron X MLP

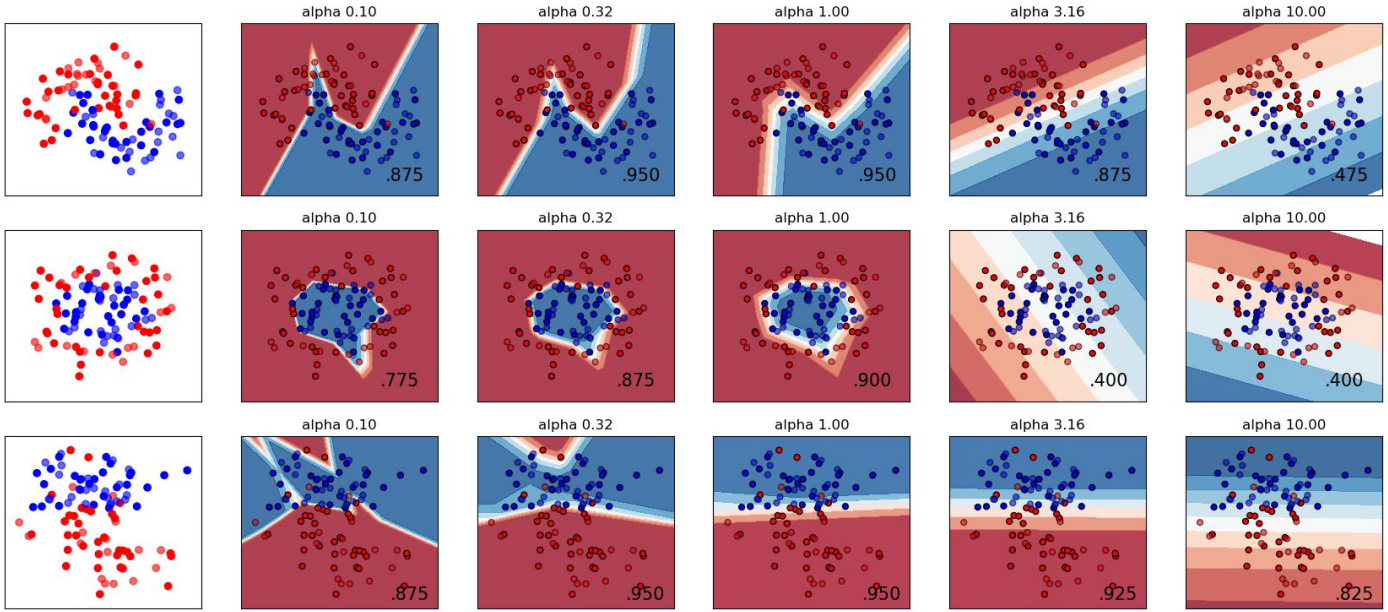






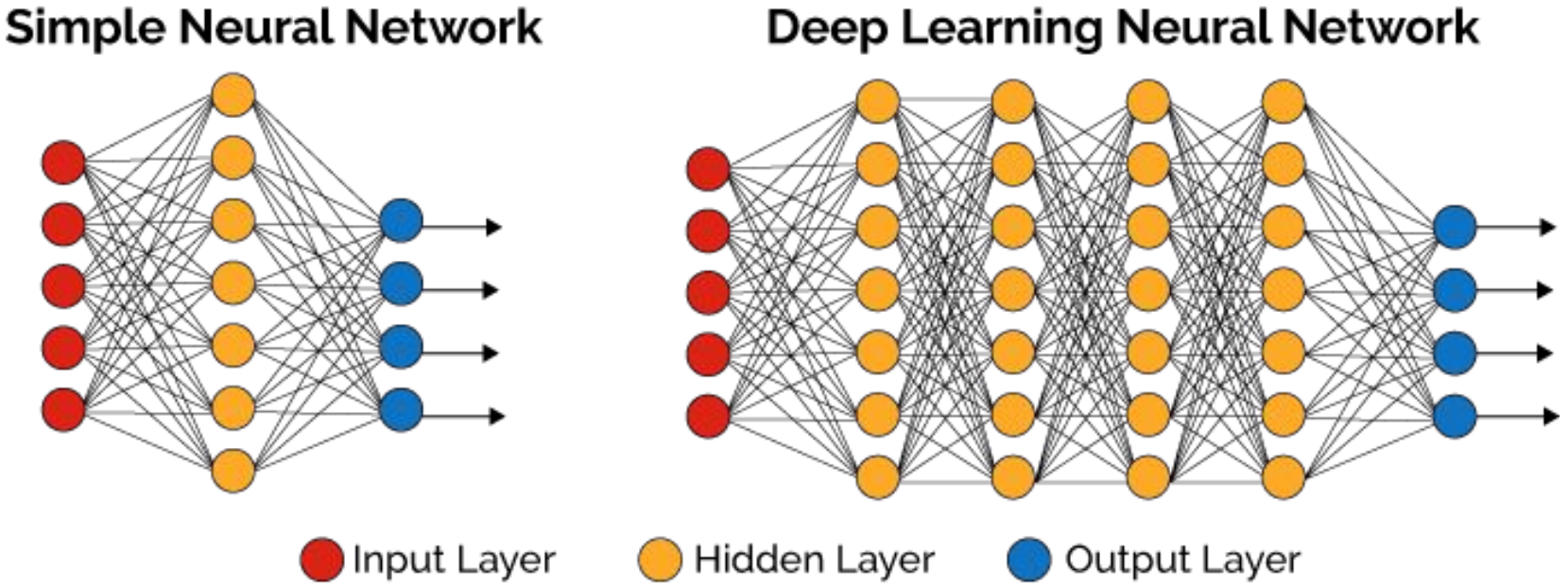
Source: https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Single Perceptron X MLP



Source: https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Multilayer Perceptron → Deep Neural Networks

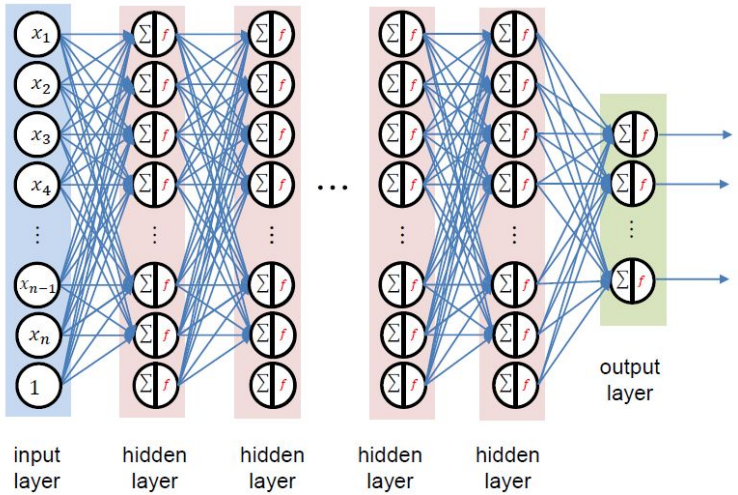


Multilayer Perceptron → Deep Neural Networks

- **Feed forward neural network**
- Neural networks “inspired by biology”

Core idea: concatenate multiple simple mappings to get one powerful mapping

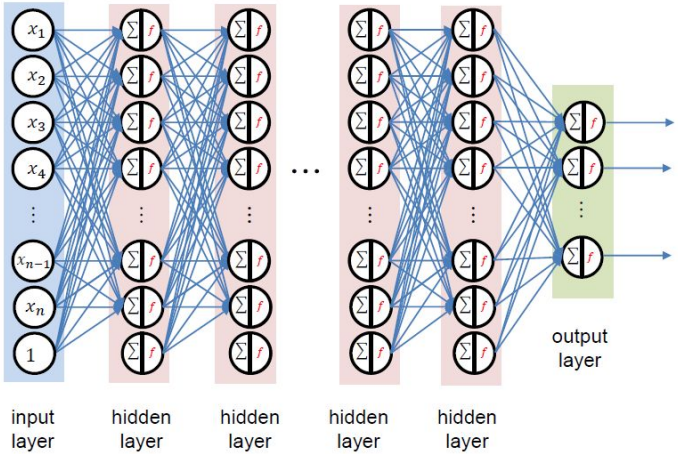
- Multiple simple steps more powerful than one complex step



Adapted from Raul Feitosa (PUC-Rio)

Multilayer Perceptron → Deep Neural Networks

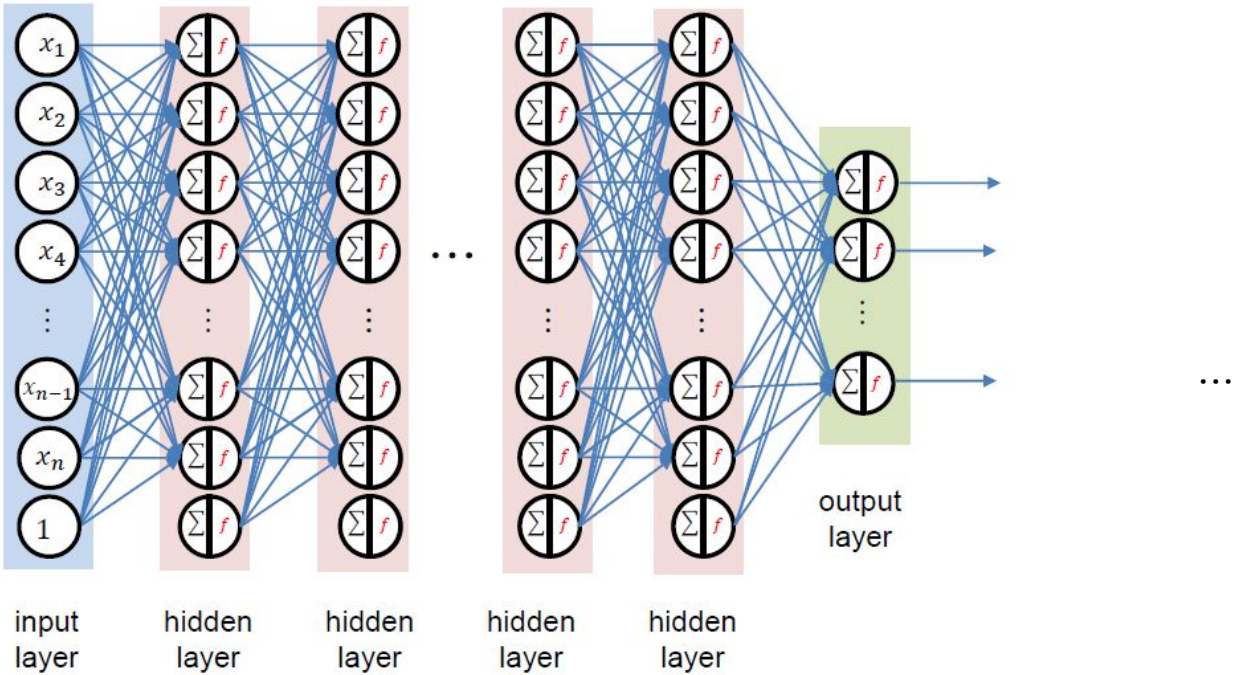
- Each neuron is fully connected to all neurons in the previous layer
- Neurons in a single layer are independent (do not share any connections)
- Consider a MLP for classifying a 200 × 200 RGB image. The **number of parameters** in a single fully connected layer will be



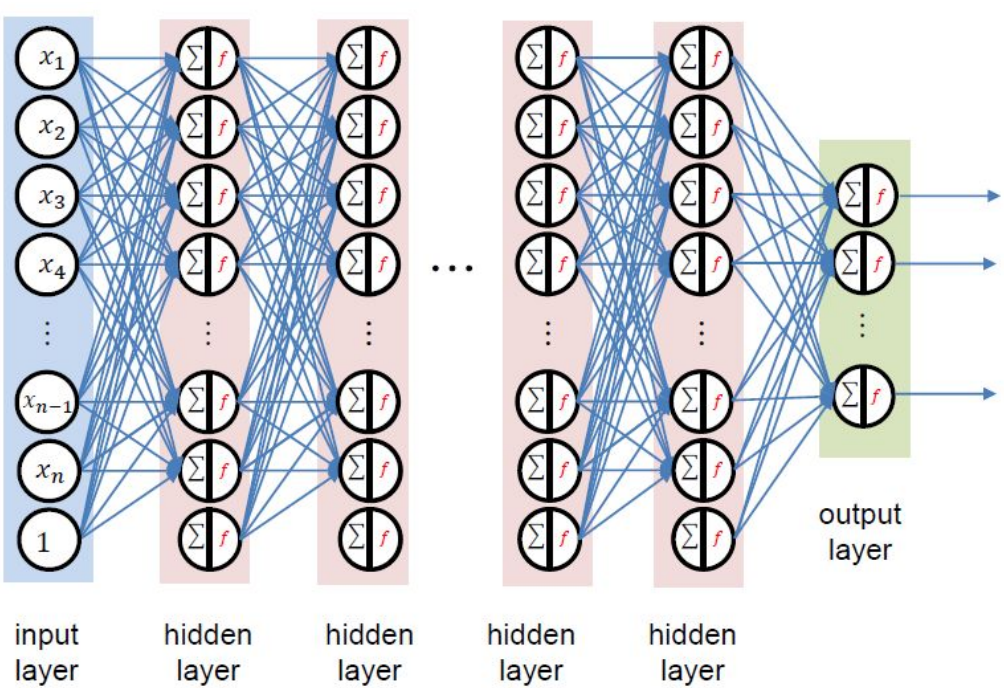
$$14.4 \times 10^9$$

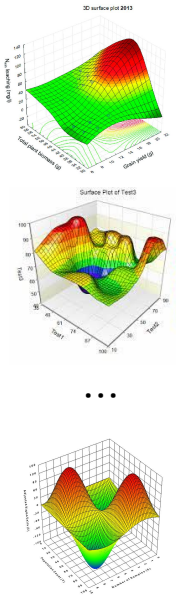
Adapted from Raul Feitosa (PUC-Rio)

How to train a MLP?



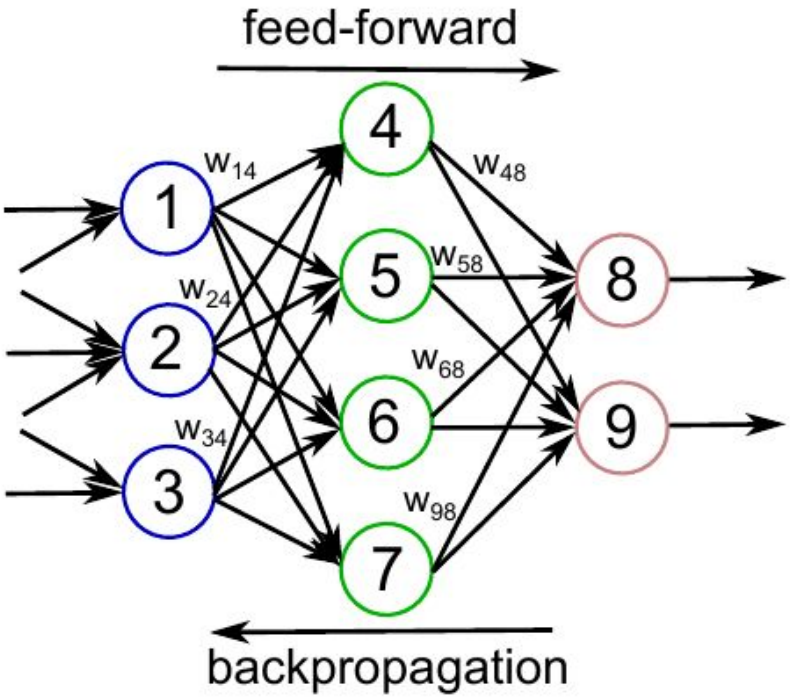
How to train a MLP?





Train the model using **gradient descent** to minimize classification error.

Two-Step Training Process

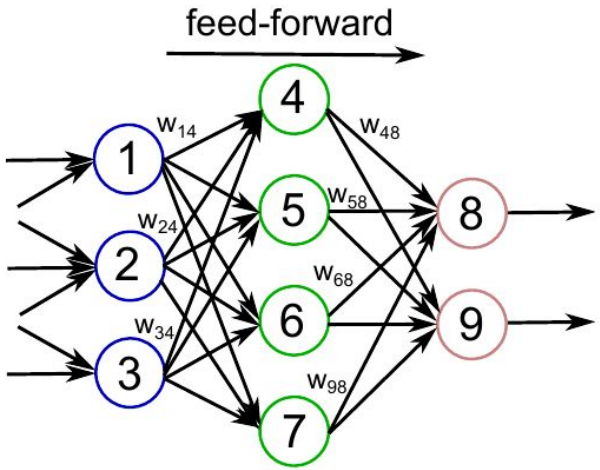


Forward Propagation (or forward pass)

- Calculation and storage of intermediate variables (including outputs) for a neural network.
- From the input layer to the output layer.

$$z = \sum_i w_i x_i + b$$

$$a = f(z)$$

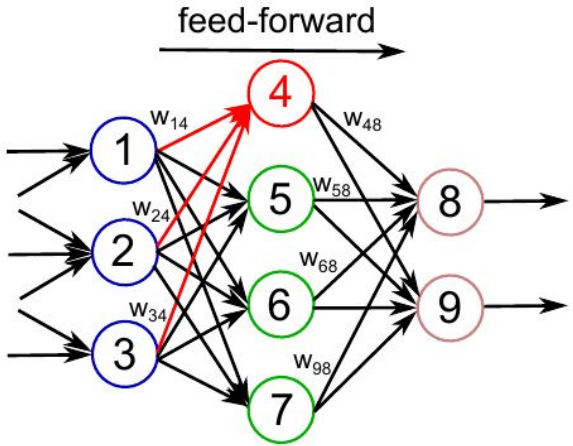


Forward Propagation (or forward pass)

- Calculation and storage of intermediate variables (including outputs) for a neural network.
- From the input layer to the output layer.

$$z^4 = W^{14}a^1 + W^{24}a^2 + W^{34}a^3 + b^4$$

$$a^4 = f(z^4)$$

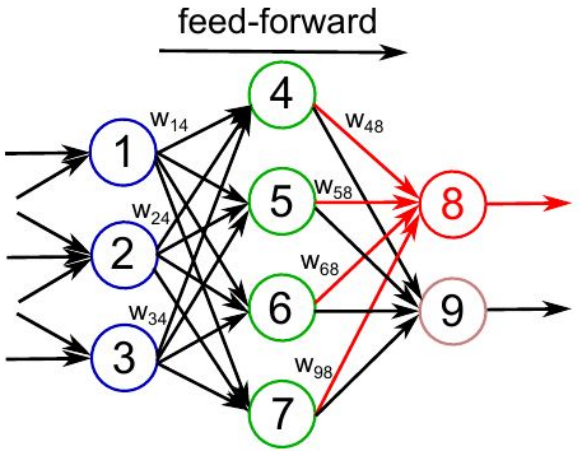


Forward Propagation (or forward pass)

- Calculation and storage of intermediate variables (including outputs) for a neural network.
- From the input layer to the output layer.

$$z^8 = W^{48}a^4 + W^{58}a^5 + W^{68}a^6 + W^{78}a^7 + b^8$$

$$a^8 = f(z^8)$$

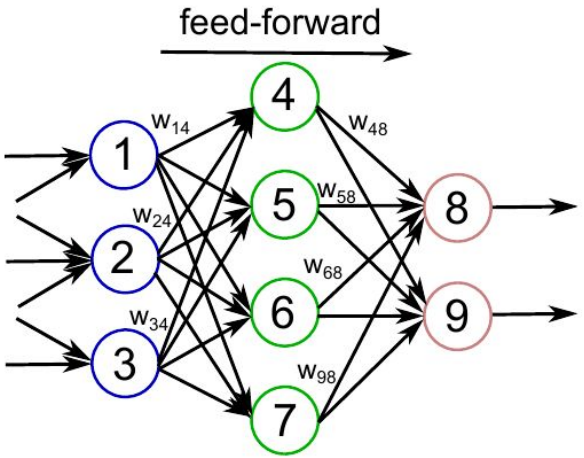


Forward Propagation (or forward pass)

- Calculation and storage of intermediate variables (including outputs) for a neural network.
- From the input layer to the output layer.

$$z^{(L+1)} = W^{(L)} a^{(L)} + b^{(L)}$$

$$a^{(L+1)} = f(z^{(L+1)})$$



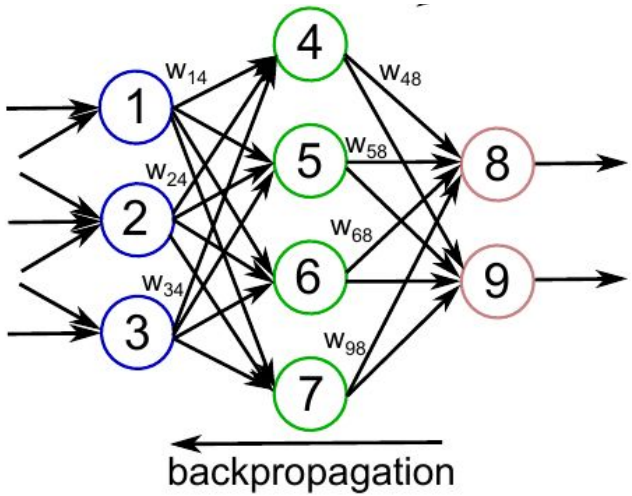
Backpropagation Algorithm

- Calculating the gradient of neural network parameters.
- The method traverses the network in reverse order, from the output to the input layer, according to **the chain rule from calculus**.

Chain rule:

- allows to find the derivative of a composite function
- helps to find the rate of change of a function within another function

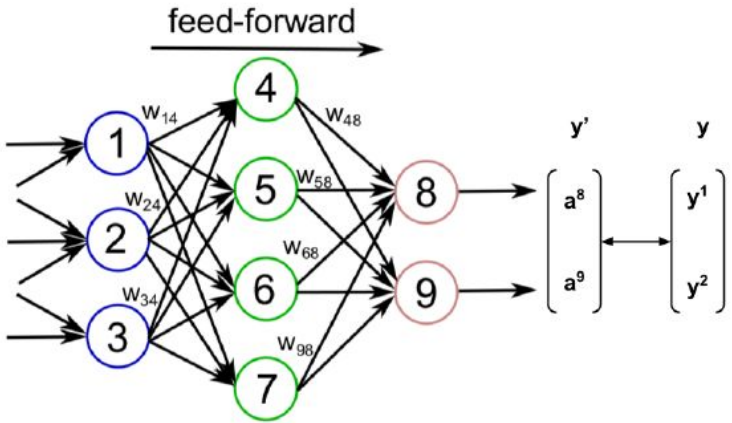
$$\frac{\partial Z}{\partial X} = \text{prod}\left(\frac{\partial Z}{\partial Y}, \frac{\partial Y}{\partial X}\right)$$



Backpropagation Algorithm - Step by Step

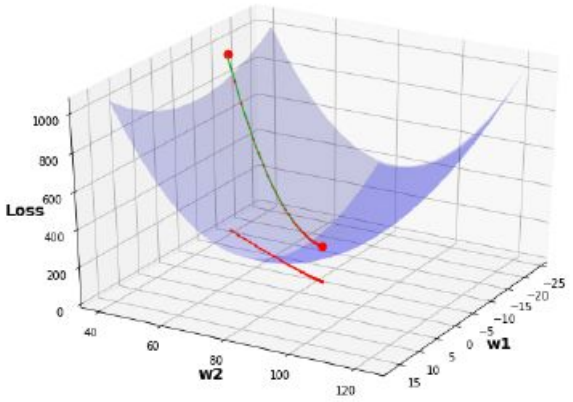
1 - Compute the loss for the network output

$$\mathcal{L}(W, b; x, y) = \frac{1}{2}\|y - a^{(L)}\|^2$$



Backpropagation Algorithm - Step by Step

1 - Compute the loss for the network output



$$\frac{\partial \mathcal{L}}{\partial W^{(L)}} = a^{(L-1)} a^{(L)}(1 - a^{(L)})(y - a^{(L)})$$

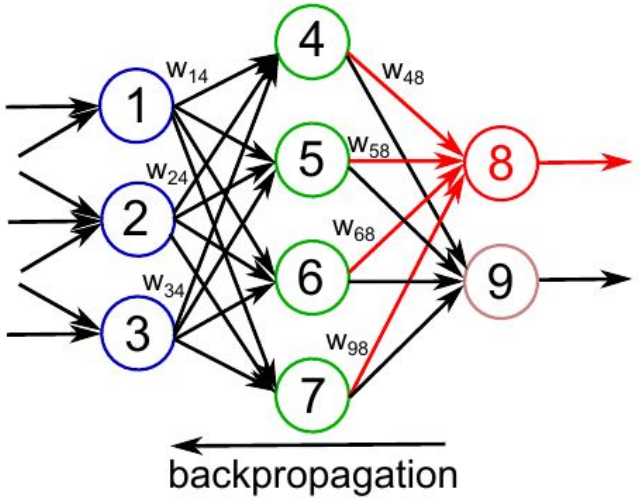
$$\delta^{(L)} = a^{(L)}(1 - a^{(L)})(y - a^{(L)})$$

$$\frac{\partial \mathcal{L}}{\partial W^{(L)}} = a^{(L-1)} \delta^{(L)}$$

Backpropagation Algorithm - Step by Step

2 - Propagating the error to neurons in the last layer

$$\delta^8 = a^8(1 - a^8)(y - a^8)$$



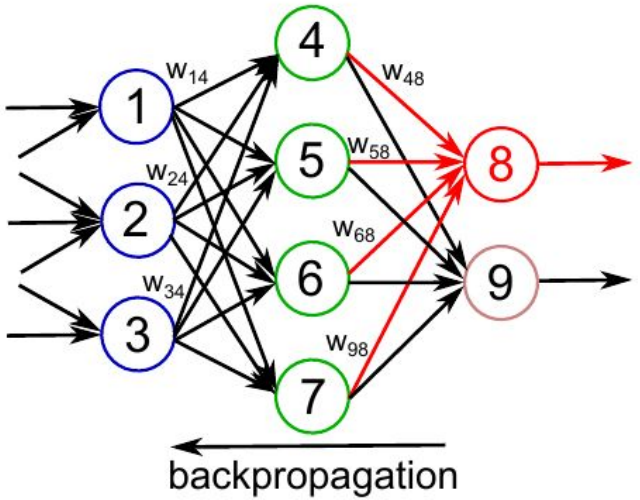
Backpropagation Algorithm - Step by Step

2 - Propagating the error to neurons in the last layer

$$W^{(L)} = W^{(L)} - \alpha \frac{\partial \mathcal{L}}{\partial W^{(L)}}$$

$$\frac{\partial \mathcal{L}}{\partial W^{(L)}} = a^{(L-1)} \delta^{(L)}$$

$$W_{48} = W_{48} - \alpha a^4 \delta^8$$
$$W_{58} = W_{58} - \alpha a^5 \delta^8$$
$$W_{68} = W_{68} - \alpha a^6 \delta^8$$
$$W_{78} = W_{78} - \alpha a^7 \delta^8$$



Backpropagation Algorithm - Step by Step

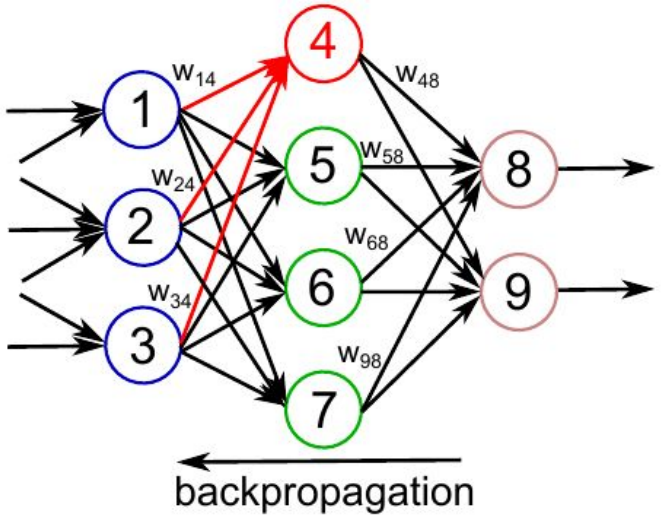
3 - Propagating the error to hidden layers

$$\delta^{(L-1)} = (\sum_i \delta_i^{(L)} w_i^{(L)}) f'(z^{(L-1)})$$

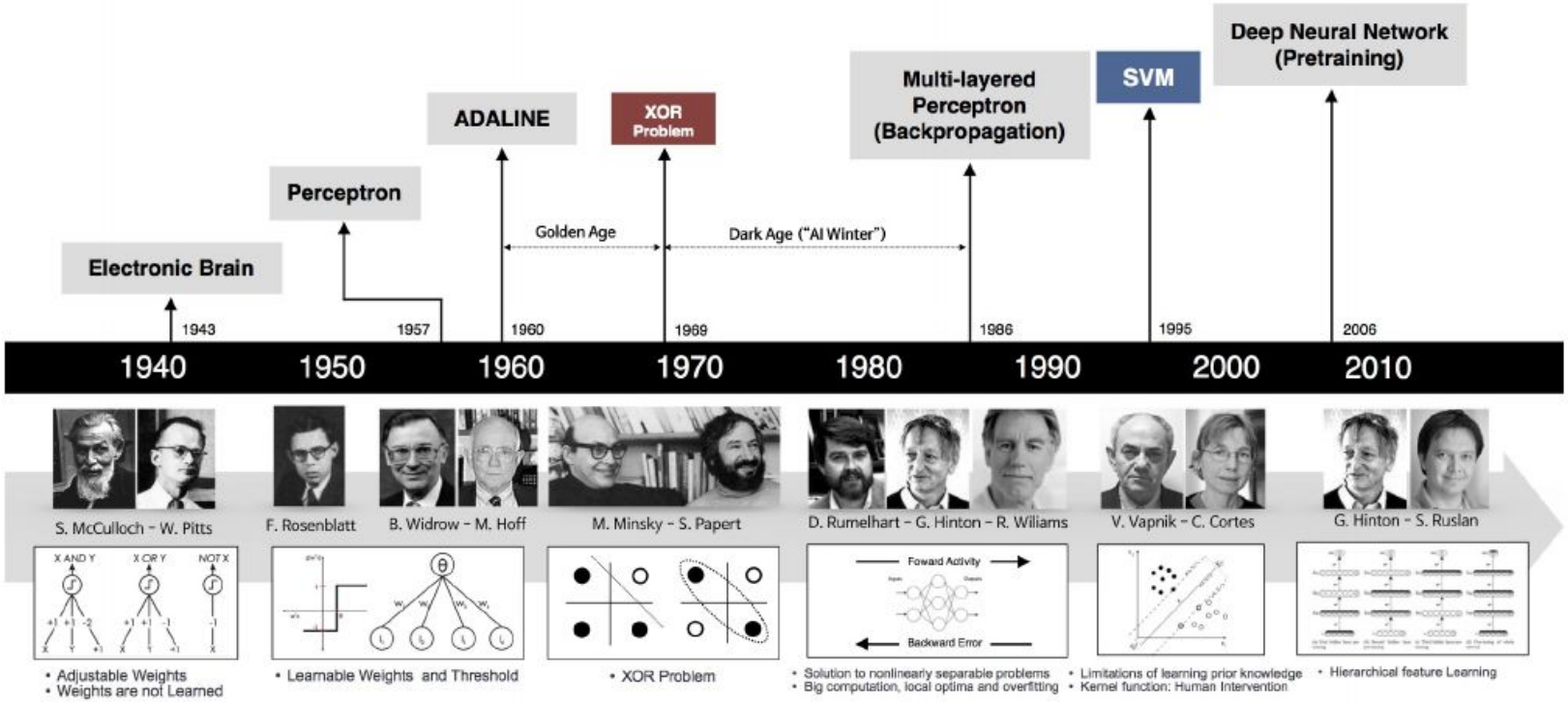
$$\delta^4 = (\delta^8 W_{48} + \delta^9 W_{49}) a^4 (1 - a^4)$$

$$W_{14} = W_{14} - \alpha \delta^4 a^1$$
$$W_{24} = W_{24} - \alpha \delta^4 a^2$$
$$W_{34} = W_{34} - \alpha \delta^4 a^3$$

$$\delta^4 = (\delta^8 W_{48} + \delta^9 W_{49}) a^4 (1 - a^4)$$
$$b_4 = b_4 - \alpha \delta^4 * 1$$



Artificial Neural Networks - Historical Context

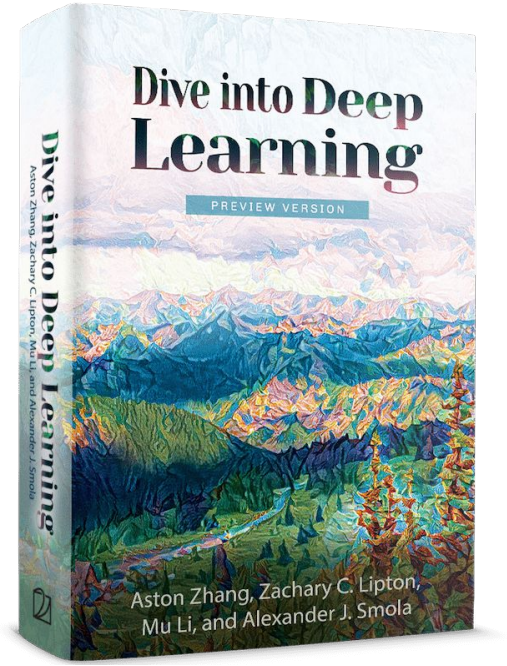


Recommended Reading

Book: **Dive into Deep Learning. [Online].**

Available: https://d2l.ai/

- Chapter 1 - Introduction: [link]
- Chapter 4 - Linear Neural Networks for Classification: [link]
- Chapter 5 - Multilayer Perceptrons: [link]



Recommended Videos

Gradient descent:
https://youtu.be/IHZwWFHWa-w

Backpropagation algorithm:

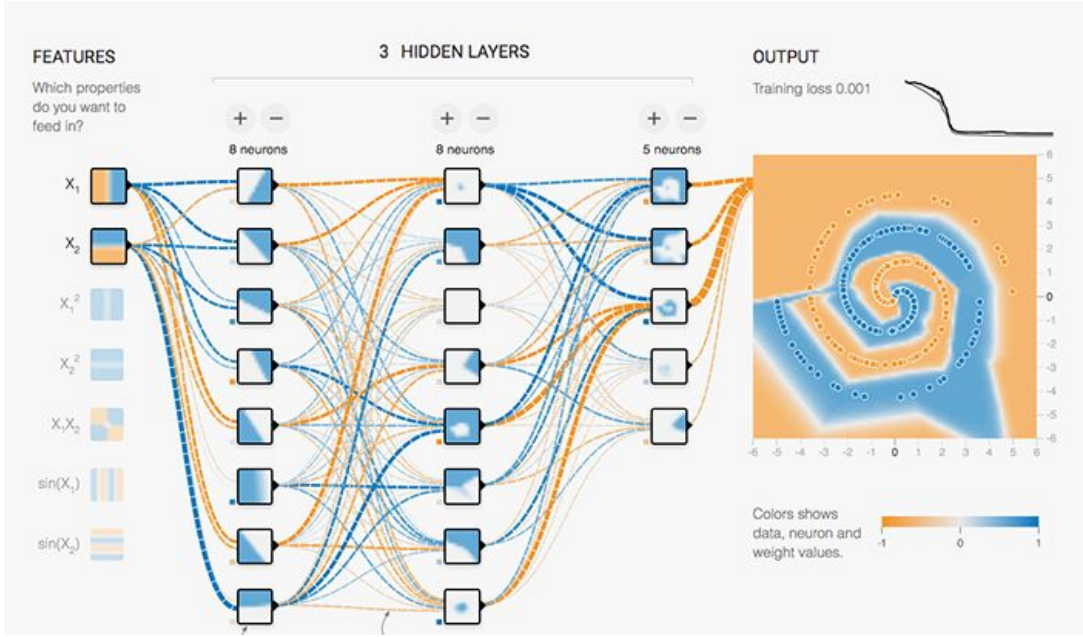
https://youtu.be/Ilg3gGewQ5U

Tools

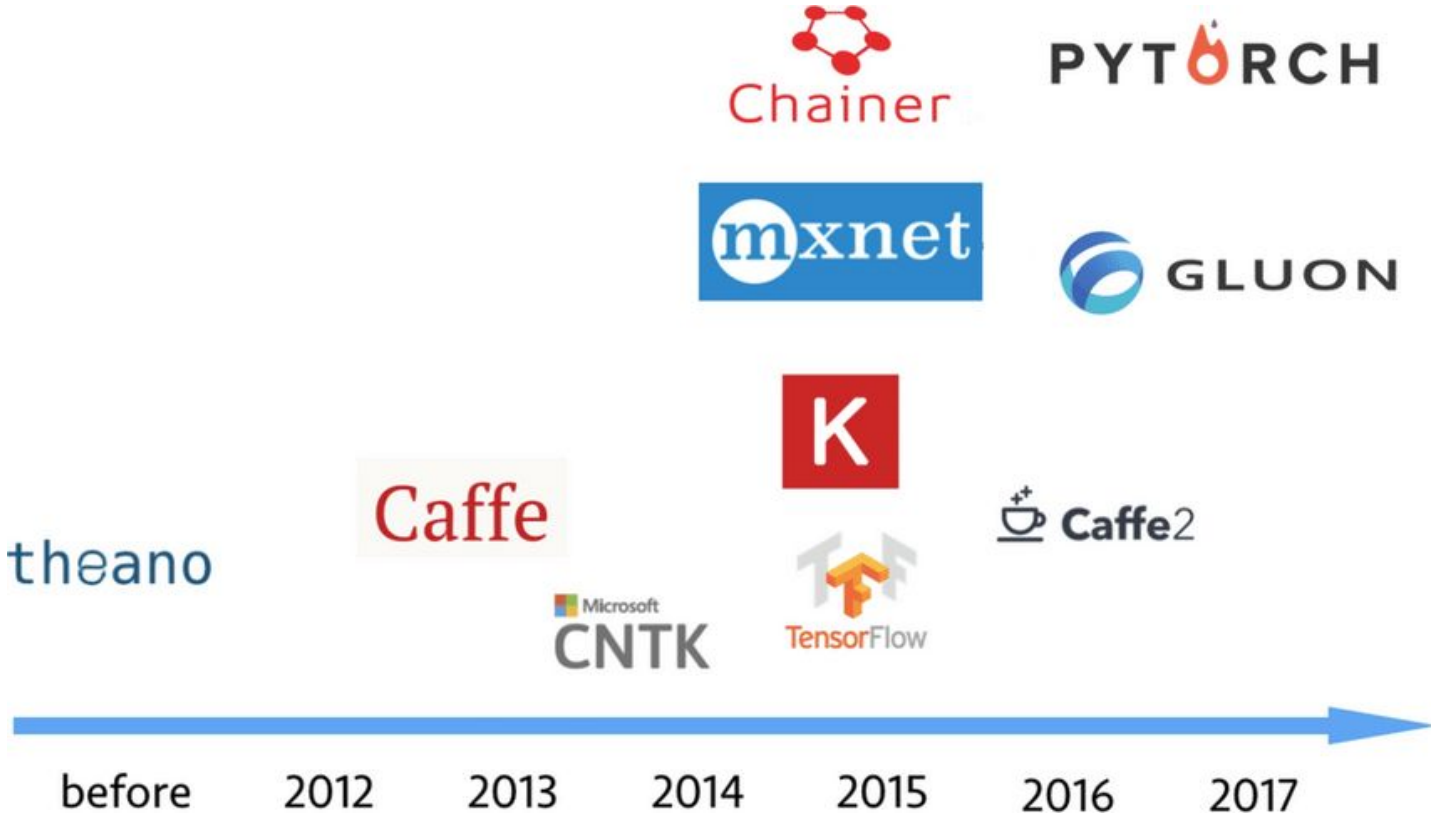
- Python:
- Pytorch: https://pytorch.org/get-started/locally/
- Google Colab
- Sheffield HPC
- https://machinelearningmastery.com/pytorch-tutorial-develop-deep-learning-models/

Playground Tensor Flow

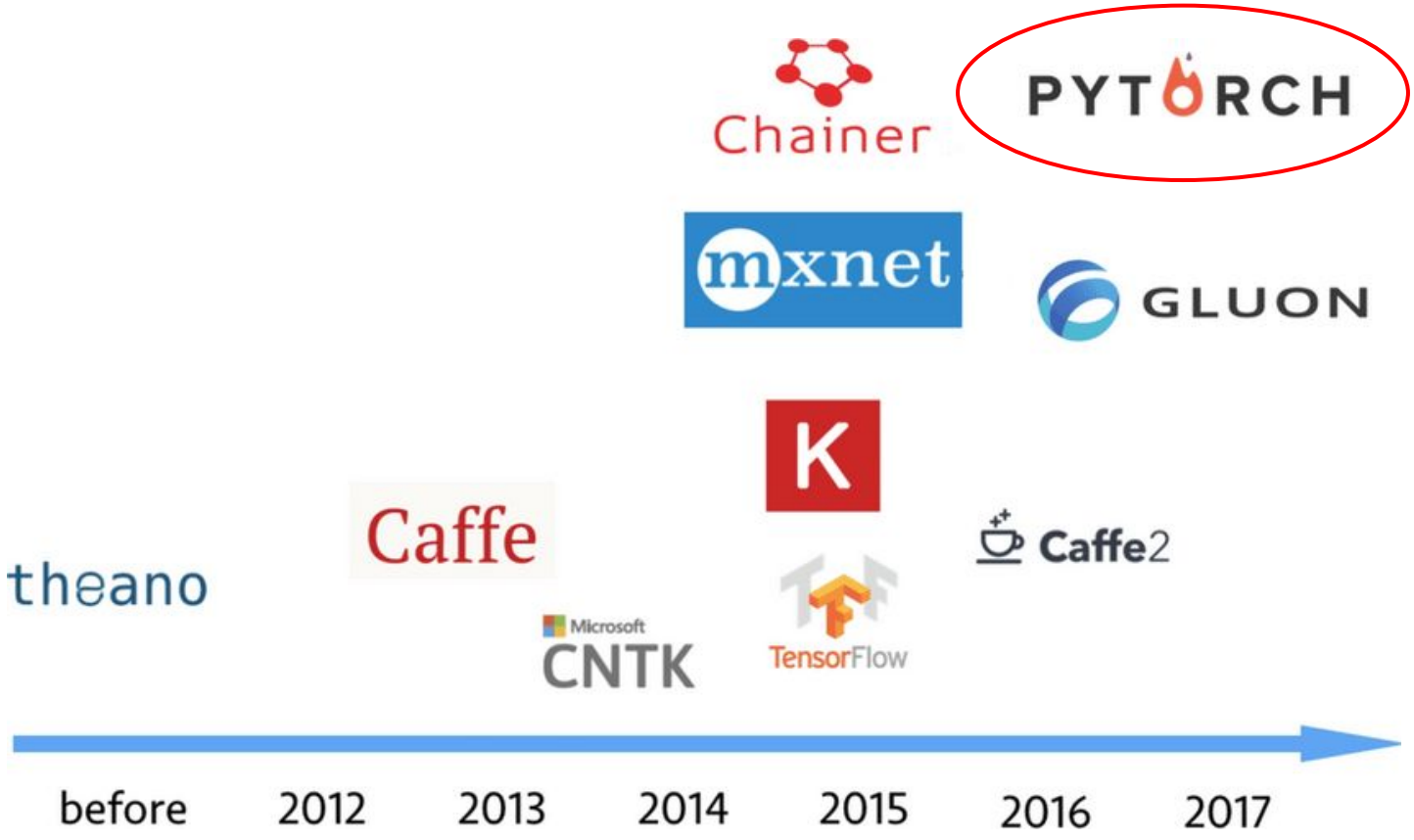
https://playground.tensorflow.org



Deep Learning Frameworks



https://en.wikipedia.org/wiki/Comparison_of_deep_learning_software

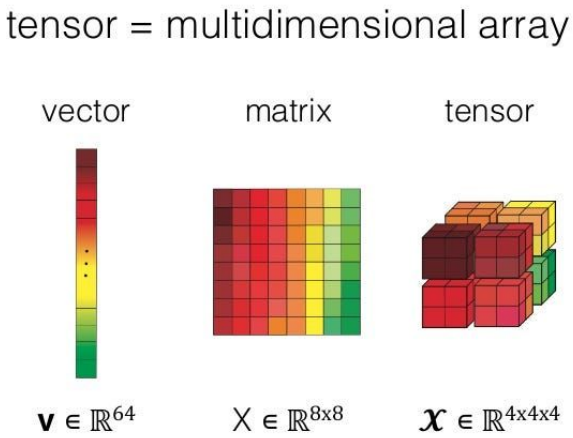


https://en.wikipedia.org/wiki/Comparison_of_deep_learning_software

Tensors

What is a Tensor?

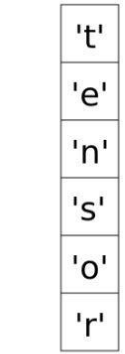
- A fundamental data structure that represents multi-dimensional arrays of numerical values
- The primary building blocks used to store and manipulate data in neural networks
- Generalize scalars, vectors, and matrices to higher dimensions, making them suitable for handling complex data such as images, videos, and text



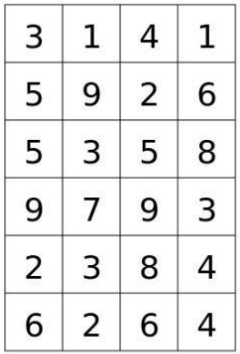
Syntax similar to ndarrays (Numpy)

Type of Tensors

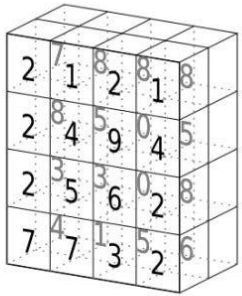
Data type	dtype	CPU tensor	GPU tensor
32-bit floating point	`torch.float32` or `torch.float`	`torch.FloatTensor`	`torch.cuda.FloatTensor`
64-bit floating point	`torch.float64` or `torch.double`	`torch.DoubleTensor`	`torch.cuda.DoubleTensor`
16-bit floating point	`torch.float16` or `torch.half`	`torch.HalfTensor`	`torch.cuda.HalfTensor`
8-bit integer (unsigned)	`torch.uint8`	`torch.ByteTensor`	`torch.cuda.ByteTensor`
8-bit integer (signed)	`torch.int8`	`torch.CharTensor`	`torch.cuda.CharTensor`
16-bit integer (signed)	`torch.int16` or `torch.short`	`torch.ShortTensor`	`torch.cuda.ShortTensor`
32-bit integer (signed)	`torch.int32` or `torch.int`	`torch.IntTensor`	`torch.cuda.IntTensor`
64-bit integer (signed)	`torch.int64` or `torch.long`	`torch.LongTensor`	`torch.cuda.LongTensor`
Boolean	`torch.bool`	`torch.BoolTensor`	`torch.cuda.BoolTensor`



tensor of dimensions [6]
(vector of dimension 6)



tensor of dimensions [6,4]
(matrix 6 by 4)



tensor of dimensions [4,4,2]

PyTorch provides functions for translating ndarrays to tensors and vice versa

PyTorch

- Python
- Off-the-shelf models
- Efficient, simple and with several functionalities
- Very good integration with other Python libraries (e.g. Numpy/Scipy)
- Dynamic Graphs

https://pytorch.org/

PyTorch

- Python
- Off-the-shelf models
- Efficient, simple and with several functionalities
- Very good integration with other Python libraries (e.g. Numpy/Scipy)
- Dynamic Graphs

- Typically refers to a computational graph that can be built and modified during runtime.
- A computational graph or dataflow graph, is a visual representation of mathematical computations or operations as nodes connected by edges.

https://pytorch.org/

PyTorch - Dynamic Graphs

A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))
```

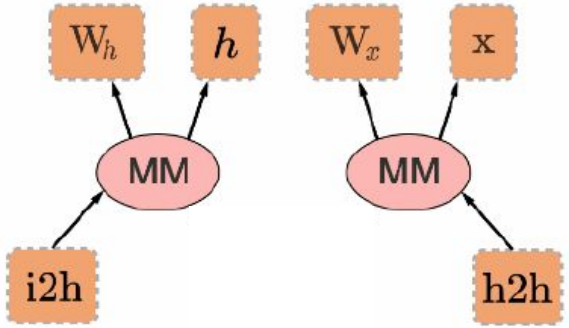
PyTorch - Dynamic Graphs

A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))

i2h = torch.mm(W_x, x.t())
h2h = torch.mm(W_h, prev_h.t())
```



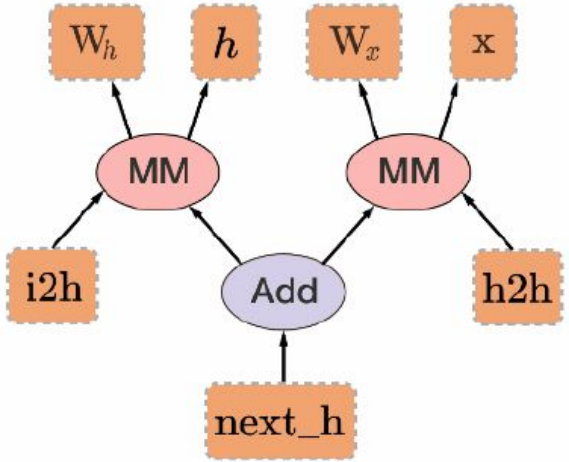
PyTorch - Dynamic Graphs

A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))

i2h = torch.mm(W_x, x.t())
h2h = torch.mm(W_h, prev_h.t())
next_h = i2h + h2h
```



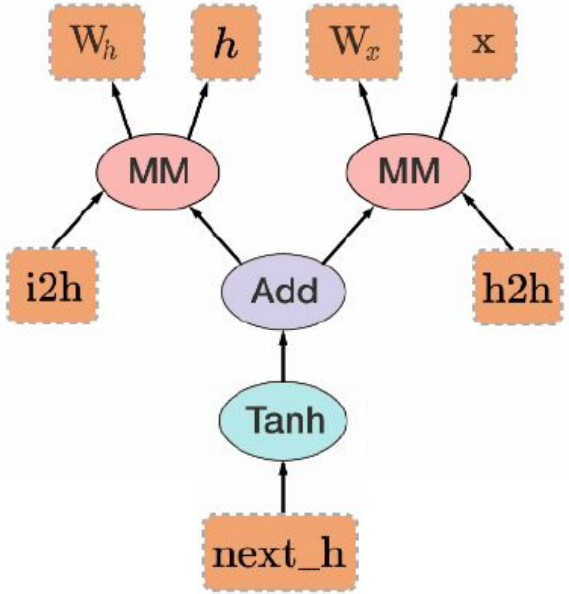
PyTorch - Dynamic Graphs

A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))

i2h = torch.mm(W_x, x.t())
h2h = torch.mm(W_h, prev_h.t())
next_h = i2h + h2h
next_h = next_h.tanh()
```



PyTorch - Dynamic Graphs

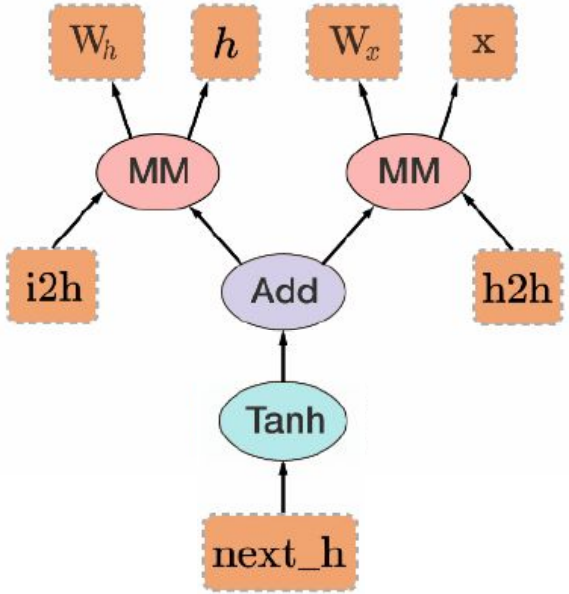
A graph is created on the fly

```
from torch.autograd import Variable

x = Variable(torch.randn(1, 10))
prev_h = Variable(torch.randn(1, 20))
W_h = Variable(torch.randn(20, 20))
W_x = Variable(torch.randn(20, 10))

i2h = torch.mm(W_x, x.t())
h2h = torch.mm(W_h, prev_h.t())
next_h = i2h + h2h
next_h = next_h.tanh()

next_h.backward(torch.ones(1, 20))
```



Backpropagation uses the dynamically built graph!

Training Procedure

Basic Steps

1. Data Loader
2. Neural Network Architecture
3. Optimizer
4. Loss Function